

Notas de Programación Competitiva

Milena Pellegrino

2025

Prefacio

Tenia este espacio al pedo y no sabia que escribir, le puse prefacio para que parezca profesional pero dice huevadas, despues se me ocurrio escribir un miini template que no tenga tantas cosas como mi template pero me quedaba grande la hoja y ademas ya habia gastado tiempo en escribir este hermoso prefacio, asi que simplemente deje esto, y puse el template abajo.

```
#include<bits/stdc++.h>
using namespace std;

#define fore(i,a,b) for(ll i=(a);i<(b);i++)
#define all(x) (x).begin(),(x).end()
#define pb push_back
#define fst first
#define snd secondt
#define sz(x) ((int)x.size())
#define db(x) cout<<#x<< " = " <<(x)<<"\n"

#define FIN ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);

using ll = long long;
typedef vector<ll> vi;

int main(){
    FIN;

    return 0;
}
```

Listing 1: Mi mini template bro

Me quedo este espacio tambien libre y quise escribir esto para parecer toda una profesional, pero no se me ocurrio algo interesante, asi que, que le dice el 1 al 10? para ser como yo tienes que ser sincero.

Índice general

1. Introducción	1
1.1. Sobre las notas	1
1.2. Cosas de C++	1
1.2.1. Tamaño de los tipos de datos	1
1.2.2. Lectura de líneas con espacios en C++	1
1.2.3. Conversiones	1
String a Numero y de Numero a String	1
Char a Numero y de Numero a Char	2
Char a Posición Alfabética y Viceversa	2
Mayúscula a Minúscula y Viceversa	3
Char, ASCII y Viceversa	3
String y Char	3
String a Vector de Chars y Viceversa	3
Numero Decimal a Otras Bases	4
String en Base N a Decimal	4
Numeros entre Tipos	4
String a Array de Digits	5
1.3. Cosas comunes en programación competitiva	5
1.3.1. Notación BIG O	5
1.3.2. Resize() en c++	5
2. Libreria Estandar C++ (STD)	7
2.1. Queue (cola)	7
2.2. Stack (pila)	7
2.3. Set (conjunto)	8
2.4. Enteros de 128 bits	9
2.4.1. Entero con signo	9
2.4.2. Entero sin signo	9
2.4.3. Notas importantes	9
2.5. Representación Binaria en C++	9
2.5.1. Mostrar un número en binario	9
2.5.2. Obtener la representación binaria como cadena	10
2.5.3. Conversión binaria manual	10
3. Búsquedas	11
3.1. Búsqueda binaria	11
3.1.1. Función lower_bound	11
3.2. Problema de ejemplo	11

3.2.1.	Descripción	11
3.2.2.	Solución	11
3.2.3.	Código de Solución	12
3.3.	Uso de <code>lower_bound</code> para obtener índices	12
3.3.1.	Obtención del índice	12
3.3.2.	Problema	13
3.4.	Búsqueda binaria en la respuesta	14
3.4.1.	Caso general	14
3.4.2.	Ejemplo 1: Problema de Sumas	15
3.4.3.	Ejemplo 2: Problema de Porcentajes	15
3.4.4.	Ejemplo 3: Problema de Recursos	16
3.4.5.	Consideraciones Importantes	17
3.4.6.	Complejidad	17
3.5.	Comparadores personalizados	17
3.5.1.	Comparadores en <code>sort</code>	18
	Orden inverso con <code>greater<T></code>	18
	Comparador con lambda	18
	Comparador como struct (necesario para <code>set</code> y <code>map</code>)	18
3.5.2.	<code>lower_bound</code> y <code>upper_bound</code> con comparador	18
	Equivalencias útiles	19
3.5.3.	<code>lower_bound</code> en <code>set</code> y <code>map</code>	19
3.5.4.	Comparadores para pares	19
3.5.5.	Trampas comunes	20
4.	Matemática	21
4.1.	Operaciones basicas	21
4.1.1.	Obtener π	21
4.1.2.	Obtener la raíz cuadrada	21
4.1.3.	Máximo Común Divisor (MCD) y Mínimo Común Múltiplo (MCM)	21
4.2.	Reglas de Divisibilidad	22
4.2.1.	Reglas de Divisibilidad Básicas	23
4.2.2.	Reglas de Divisibilidad Menos Comunes	23
4.2.3.	Reglas de Divisibilidad para Números Primos Mayores	24
4.2.4.	Métodos Generales para Verificar Divisibilidad	24
4.2.5.	Generación de divisores	24
4.3.	Números Primos	24
4.3.1.	Chequear si un número es primo	24
4.3.2.	Criba de Eratóstenes para factorizar números	25
4.4.	Permutaciones	26
4.4.1.	Computar las permutaciones	26
4.5.	Combinatoria	26
4.5.1.	Triángulo de Pascal	26
4.6.	Geometría	28
4.6.1.	Distancia entre dos puntos	28
4.6.2.	Triángulos	28
	Ley de Cosenos	28
	Ley de Senos	29
4.7.	Transformada Rápida de Fourier (FFT)	29

4.7.1.	Idea general	29
4.7.2.	Transformada Discreta de Fourier (DFT)	29
4.7.3.	Multiplicación de polinomios con DFT	30
4.7.4.	Algoritmo: FFT recursiva (Cooley-Tukey)	30
4.7.5.	Función <code>multiply</code> : convolución de polinomios	31
4.7.6.	Convolución para contar coincidencias	32
4.7.7.	Consideraciones importantes	32
4.7.8.	Complejidad	32
5.	Programacion Dinamica (dp)	33
5.1.	De una función recursiva por partes a código con DP	33
5.2.	Ejemplos clásicos	34
5.2.1.	A - Frog 1	34
5.2.2.	C - Vacation	35
5.2.3.	D - Knapsack 0/1	35
5.2.4.	G - Longest Path in DAG	36
5.3.	Resumen	36
6.	Grafos	37
6.1.	Representaciones comunes	37
6.2.	Recorridos clásicos	37
6.2.1.	DFS (Depth-First Search)	37
6.2.2.	BFS (Breadth-First Search)	38
6.3.	Componentes conexas	40
6.4.	Dijkstra (camino mínimos)	41
6.5.	Bellman-Ford	41
6.6.	Problemas clásicos	42
6.7.	¿Cómo verificar si un grafo es una línea (path)?	42
6.8.	Detectar ciclos en grafo no dirigido	43
6.9.	Detectar ciclos en grafo dirigido	44
6.10.	Verificar si hay ciclo impar	45
6.11.	Verificar si el grafo es bipartito	46
6.12.	Verificar si el grafo es un árbol	46
6.13.	Operaciones básicas en árboles	47
6.14.	Número de hojas en un árbol	48
6.15.	Diámetro de un árbol	49
6.16.	Orden Topológico	51
6.17.	Componentes conexas	53
6.18.	Matching (Algoritmo Húngaro)	54
7.	Strings	59
7.1.	Permutaciones de string	59
7.2.	Palindromic Tree (Eertree)	59
7.2.1.	¿Qué es?	59
7.2.2.	Estructura interna	60
7.2.3.	Implementación	60
7.2.4.	¿Qué tengo que pasarle?	61
7.2.5.	¿Qué puedo consultar?	61
7.2.6.	Ejemplo de aplicación	61

7.2.7. Complejidad	62
8. Estructuras de Datos	63
8.1. Union Find / Disjoint Set Union (DSU)	63
8.2. Fenwick Tree (BIT)	64
9. Arrays / Subarrays	67
9.1. Kadane	67
9.1.1. Kadane con restricción (el subarreglo debe incluir k)	67

Capítulo 1

Introducción

1.1. Sobre las notas

Estas notas explican recordatorios y cosas basicas en programación competitiva. Fueron escritas con la idea de usarla como notebook en competencias.

1.2. Cosas de C++

1.2.1. Tamaño de los tipos de datos

- **Tamaño de un long long:** Es un número de 64 bits, 2^{64} o aproximadamente 10^{19} .

1.2.2. Lectura de líneas con espacios en C++

En C++, para leer una línea completa que puede contener espacios (como una oración o una entrada con varios números separados), se utiliza:

```
getline(std::cin, s);
```

Sin embargo, si antes se usó

```
std::cin >> variable;
```

entonces `getline()` puede leer una línea vacía. Para evitarlo, se debe limpiar el buffer con:

```
std::cin.ignore();
```

```
cin.ignore();
```

1.2.3. Conversiones

String a Numero y de Numero a String

- De string a int: `int x = stoi("123");`
- De string a long long: `ll x = stoll("123456789");`
- De string a double: `double x = stod("3.14");`

- De string a float: `float x = stof("3.14");`
- De string a long double: `long double x = stold("3.14");`
- De numero a string: `string s = to_string(x);`

Char a Numero y de Numero a Char

De char numeral a número Si el char representa un dígito ('0' a '9'):

```
int x = c - '0';
```

Ejemplo:

```
char c = '7';
int x = c - '0'; // 7
```

De número a char numeral

```
char c = x + '0';
```

Ejemplo:

```
int x = 7;
char c = x + '0'; // '7'
```

Char a Posición Alfabética y Viceversa

Minúscula a número ($a \rightarrow 1, b \rightarrow 2, \dots$)

$$\text{posición}(c) = c - 'a' + 1$$

```
int pos = c - 'a' + 1;
```

Número a minúscula ($1 \rightarrow a, 2 \rightarrow b, \dots$)

```
char c = pos - 1 + 'a';
```

Mayúscula a número ($A \rightarrow 1, B \rightarrow 2, \dots$)

$$\text{posición}(C) = C - 'A' + 1$$

```
int pos = c - 'A' + 1;
```

Número a mayúscula ($1 \rightarrow A, 2 \rightarrow B, \dots$)

```
char c = pos - 1 + 'A';
```

Mayúscula a Minúscula y Viceversa

■ Convertir a mayúscula

```
char c = 'b';  
c = toupper(c); // 'B'
```

■ Convertir a minúscula

```
char c = 'G';  
c = tolower(c); // 'g'
```

■ Verificar si es mayúscula

```
isupper(c)
```

■ Verificar si es minúscula

```
islower(c)
```

Char, ASCII y Viceversa

Char a código ASCII

```
int ascii = (int)c;
```

ASCII a char

```
char c = (char)x;
```

Ejemplo:

```
char c = 'A';  
int x = (int)c; // 65  
  
char d = (char)65; // 'A'
```

String y Char

Char a string

```
char c = 'a';  
string s(1, c); // "a"
```

O también:

```
string s = string(1, c);
```

String de longitud 1 a char

```
char c = s[0];
```

String a Vector de Chars y Viceversa

String a vector<char>

```
string s = "hola";  
vector<char> v(s.begin(), s.end());
```

```
vector<char> a string  
    string s(v.begin(), v.end());
```

Numero Decimal a Otras Bases

Decimal a binario

```
ll x = 13;  
string s = bitset<64>(x).to_string();  
  
s.erase(0, s.find('1'));
```

Decimal a hexadecimal

```
stringstream ss;  
ss << hex << x;  
string hexNum = ss.str();
```

Decimal a octal

```
stringstream ss;  
ss << oct << x;  
string octNum = ss.str();
```

String en Base N a Decimal

Binario a decimal

```
ll x = stoll(binary, nullptr, 2);
```

Hexadecimal a decimal

```
ll x = stoll(hexString, nullptr, 16);
```

Octal a decimal

```
ll x = stoll(octalString, nullptr, 8);
```

Numeros entre Tipos

int a long long

```
ll x = (ll)a;
```

double a int

```
int x = (int)d;
```

int a double

```
double x = (double)a;
```

División real entre enteros

```
double ans = (double)a / b;
```

String a Array de Dìgitos

String de dígitos a vector<int>

```
string s = "12345";
vector<int> digits;

for(char c : s){
    digits.push_back(c - '0');
}
```

vector<int> de dígitos a string

```
string s;

for(int d : digits){
    s += char(d + '0');
}
```

Ordenar partes (subrangos) de un vector

En C++, se puede ordenar una parte de un vector (es decir, un subrango) utilizando la función `sort` de la STL, especificando iteradores de inicio y fin.

Esto es especialmente útil cuando se necesita dividir un conjunto en mitades o grupos, por ejemplo, para aplicar diferentes criterios de ordenamiento en cada parte.

```
// Supongamos que tenemos un vector de arreglos de 3 enteros:
vector<array<ll, 3>> v(n);

// Ordenamos solo la primera mitad del vector por el segundo campo (índice)
sort(v.begin(), v.begin() + n/2, [](const array<ll, 3> &a, const array<ll, 3> &b){
    return a[1] < b[1];
});

// Ordenamos la segunda mitad también por el segundo campo:
sort(v.begin() + n/2, v.end(), [](const array<ll, 3> &a, const array<ll, 3> &b){
    return a[1] < b[1];
});
```

Aplicación: esta técnica aparece frecuentemente en problemas donde hay que emparejar elementos de dos mitades, o donde diferentes partes del conjunto deben ser tratadas con diferentes estrategias.

1.3. Cosas comunes en programación competitiva

1.3.1. Notación BIG O

1.3.2. Resize() en c++

La función `std::vector::resize()` ajusta el número de elementos en un vector. Su acción depende del tamaño deseado:

Tamaño del input: N	Peor complejidad aceptada
$N \leq 22$ ($\approx 2 \cdot 10^1$)	$O(2^N)$
$N \leq 100$ (10^2)	$O(N^3)$
$N \leq 1000$ (10^3)	$O(N^2)$
$N \leq 1,000,000$ (10^6)	$O(N \log N)$
$N \leq 1,000,000,000$ (10^9)	$O(N)$, $O(\log N)$, $O(1)$

Cuadro 1.1: Relación entre tamaño de input y la peor complejidad aceptada.

- **Si el nuevo tamaño es mayor que el actual:** Se añaden nuevos elementos al final. Opcionalmente, se puede especificar un valor de inicialización: `mi_vector.resize(10, -1)`.
- **Si el nuevo tamaño es menor que el actual:** Se eliminan los elementos del final del vector hasta alcanzar el tamaño especificado.

```
int main() {
    int n; cin>>n;
    vector<int> numeros;
    numeros.resize(n); // Pre-asigna memoria para n elementos
    for (int i = 0; i < n; ++i) {
        cin >> numeros[i]; // Acceso directo por '\indice
    }
    return 0;
}
```

Listing 1.1: Uso eficiente de resize

Capítulo 2

Librería Estándar C++ (STD)

2.1. Queue (cola)

Las colas son estructuras first-in-first-out (FIFO).

Complejidad de todas las operaciones: $O(1)$

```
// Crear una queue:
queue<int> cola;

// Insertar elemento al FINAL de la cola:
cola.push(1)

// Ver cuantos elementos tiene:
cola.size()

// Eliminar el primer elemento:
cola.pop();

// VER cual es el primer elemento:
cola.front()

// Ver si la cola esta vacia:
if (cola.empty())
```

Listing 2.1: Funcionamiento de la queue

2.2. Stack (pila)

Las pilas son estructuras last-in-first-out (LIFO).

Complejidad de todas las operaciones: $O(1)$

```
// Crear un stack:
stack<int> pila;

// Insertar elemento al PRINCIPIO del stack:
pila.push(1);

// Ver cuantos elementos tiene:
pila.size()

// Eliminar el primer elemento:
pila.pop();
```

```
// VER cual es el primer elemento:
pila.top()

// Ver si la cola esta vacia:
if (pila.empty())
```

Listing 2.2: Funcionamiento del stack

2.3. Set (conjunto)

Almacena valores unicos de un determinado tipo, ordenados automaticamente.

```
// Crear un set:
set<int> s;

// Insertar un elemento:
s.insert(3); // O(logn)

// Ver cuantos elementos tiene: // O(1)
s.size()

// Eliminar un elemento determinado:
// Devuelve cuantos elementos borro
s.erase(1); // O(logn)

// Borra todos los elementos
us.clear();

// Verificar si un valor esta de forma rapido:
if (s.count(x)) { ... }

/*
Busca si un valor existe en el set.
Si no lo encuentra, devuelve s.end()
*/

s.find(3) // O(logn)

auto it = s.find(10);
if (it != s.end()) {
    cout << "Encontrado: " << *it << endl;
} else {
    cout << "No esta el 10\n";
}

// Otra especie de busqueda
auto it = s.lower_bound(25); // primer elemento >= 25
auto it2 = s.upper_bound(30); // primer elemento > 30

if (it != s.end()) cout << *it << endl;
if (it2 != s.end()) cout << *it2 << endl;
```

Listing 2.3: Funcionamiento del stack

Otras alternativas a set:

- `multiset`: Permite duplicados, ordenados tambien.
- `unordered_set`: Operaciones en $\mathcal{O}(1)$ promedio, pero sin orden.

2.4. Enteros de 128 bits

En GCC y Clang se puede usar `__int128` y `__uint128_t` para trabajar con enteros de 128 bits. No forman parte del estándar de C++, pero son muy útiles cuando `long long` no alcanza.

2.4.1. Entero con signo

```
using i128 = __int128_t;
```

Listing 2.4: Entero de 128 bits con signo

Rango aproximado: $[-2^{127}, 2^{127} - 1] \approx \pm 1,70 \times 10^{38}$.

2.4.2. Entero sin signo

```
using u128 = __uint128_t;
```

Listing 2.5: Entero de 128 bits sin signo

Rango: $[0, 2^{128} - 1] \approx 3,4 \times 10^{38}$.

2.4.3. Notas importantes

- `cin` y `cout` no saben leer ni imprimir `__int128` directamente.
- Si necesitas mostrarlo por pantalla, hay que convertirlo a `string` manualmente.

2.5. Representación Binaria en C++

2.5.1. Mostrar un número en binario

La forma más directa es usando `bitset` de la STL:

```
int main() {  
    int x = 13;  
    cout << bitset<15>(x) << endl; // Muestra: 000000000001101  
}
```

Listing 2.6: Muestra un número en binario con 15 bits

2.5.2. Obtener la representación binaria como cadena

Podemos obtener la representación binaria como un `string`:

```
int main() {  
    int x = 13;  
    string bin = bitset<15>(x).to_string(); // "0000000000001101"  
}
```

Listing 2.7: Convertir un número a cadena binaria

2.5.3. Conversión binaria manual

También podemos implementar la conversión binaria manualmente:

```
void mostrar_binario(int n) {  
    if (n == 0) {  
        cout << "0";  
        return;  
    }  
    string bin = "";  
    while (n > 0) {  
        bin = char('0' + (n % 2)) + bin;  
        n /= 2;  
    }  
    cout << bin << endl;  
}
```

Listing 2.8: Conversión manual de decimal a binario

Capítulo 3

Búsquedas

3.1. Búsqueda binaria

Siempre si vas a resolver algo con `lower_bound` aunque lo diga o no lo diga el problema SIEMPRE pero SIEMPRE ordenalo, me comi varios wa porque confiaba que siempre venia ordenado.

La función `lower_bound` es una herramienta fundamental en la biblioteca estándar de C++ para realizar búsquedas eficientes en contenedores ordenados.

3.1.1. Función `lower_bound`

- **Complejidad:** $O(\log n)$ para contenedores de acceso aleatorio como `vector`
- **Comportamiento:** Devuelve un iterador al primer elemento que **no es menor** que el valor dado (primer elemento $\geq$ al valor buscado)
- **Sintaxis:** `lower_bound(iterador_inicio, iterador_fin, valor)`

3.2. Problema de ejemplo

3.2.1. Descripción

El aula se representa como una línea unidimensional con celdas desde 1 hasta n . Inicialmente hay m profesores y David en posiciones distintas. En cada movimiento:

1. David se mueve a una celda adyacente o se queda
2. Todos los profesores se mueven simultáneamente a una celda adyacente o se quedan

David es atrapado cuando algún profesor está en la misma celda. Todos actúan óptimamente: David maximiza el número de movimientos hasta ser atrapado, mientras los profesores lo minimizan.

3.2.2. Solución

Para cada consulta con la posición a_i de David:

1. Encontrar los dos profesores más cercanos (izquierda y derecha)

2. Calcular la mitad de la distancia entre ellos
3. David se moverá al punto medio para maximizar el tiempo

3.2.3. Código de Solución

```

void solve() {
    ll n; cin >> n;
    ll m, q; cin >> m >> q;
    vi b(m);
    fore(i, 0, m) cin >> b[i];
    vi a(q);
    fore(i, 0, q) cin >> a[i];

    sort(all(b));

    fore(i, 0, q) {
        ll izq = 0, der = 0;
        ll res = 0;
        auto it = lower_bound(all(b), a[i]);

        if (it == b.begin()) {
            res = (*it) - 1;
        } else if (it == b.end()) {
            --it;
            res = n - (*it);
        } else {
            der = *it;
            --it;
            izq = *it;
            ll dist_der = der - a[i];
            ll dist_izq = a[i] - izq;
            res = (dist_der + dist_izq) / 2;
        }
        pri(res);
    }
}

```

Listing 3.1: Solución del problema

3.3. Uso de lower_bound para obtener índices

3.3.1. Obtención del índice

Cuando se utiliza `lower_bound` sobre un contenedor, podemos obtener la posición (índice) del elemento encontrado mediante aritmética de punteros:

```

void solve() {
    ll n; cin >> n; ll k; cin >> k;
    vi a(n); fore(i, 0, n) cin >> a[i];

    while(k--) {
        ll val; cin >> val;
        auto it = lower_bound(all(a), val);
    }
}

```

```

        ll res = it - a.begin(); // Obtenemos el indice
        pri(res + 1); // +1 para indice base 1
    }
}

```

Listing 3.2: Obtención del índice con lower_bound

3.3.2. Problema

Tenemos un edificio de n pisos, cada piso i con a_i departamentos. Las cartas están numeradas linealmente desde 1 hasta la suma total de departamentos. Dado un número de carta b_j , hay que determinar:

- El **piso** donde va la carta.
- El **departamento dentro del piso**.

Ejemplo: $n = 2$, $a = [3, 5]$, carta $b = 7 \rightarrow$ Piso 2, departamento 4.

Idea de la solución 1. Construimos un vector de **prefix sums**:

$$\text{pref}[0] = a_0, \quad \text{pref}[i] = \text{pref}[i-1] + a_i \text{ para } i \geq 1$$

Cada elemento indica la cantidad total de departamentos hasta ese piso.

2. Para cada carta b_j usamos **búsqueda binaria** con lower_bound:

$$\text{piso} = \text{lower_bound}(\text{pref.begin()}, \text{pref.end()}, b_j) - \text{pref.begin}()$$

3. Calculamos el departamento dentro del piso:

$$\text{depto} = b_j - (\text{piso} > 0 ? \text{pref}[\text{piso}-1] : 0)$$

Código en C++

```

void solve() {
    ll n, m;
    cin >> n >> m;
    vector<ll> a(n), b(m);
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    // Construir prefix sums
    vector<ll> pref(n);
    pref[0] = a[0];
    for (int i = 1; i < n; i++) pref[i] = pref[i-1] + a[i];

    for (int i = 0; i < m; i++) {
        // Encontrar piso usando búsqueda binaria
        int piso = lower_bound(pref.begin(), pref.end(), b[i]) - pref.begin();
        // Departamento dentro del piso
        ll depto = b[i] - (piso > 0 ? pref[piso-1] : 0);
    }
}

```

```
cout << piso + 1 << " " << depto << "\n";
}
}
```

Ejemplo paso a paso

$$a = [3, 5, 2] \implies pref = [3, 8, 10], \quad b = [1, 4, 9]$$

Carta b_j	Piso (0-based)	Piso (1-based)	Departamento
1	0	1	1
4	1	2	1
9	2	3	1

Complejidad

- Construir prefix sums: $O(n)$
- Cada carta: $O(\log n)$ con `lower_bound`
- Total: $O(n + m \log n)$

Tips

- ‘`lower_bound`’ devuelve iterador, se resta ‘`pref.begin()`’ para obtener índice.
- Esta técnica combina **prefix sums** con **búsqueda binaria** para ubicar rápidamente rangos acumulados.

3.4. Búsqueda binaria en la respuesta

3.4.1. Caso general

```
bool can(int mid) {
    // Esta funcion debe responder si con mid como respuesta
    // es posible cumplir con la condicion del problema

    // Implementar la logica del chequeo

    return true;
}

int main(){
    // Busqueda binaria
    int low = 0, high = 1e9; // Ajustar limites segun el problema
    int ans = -1;

    while (low <= high) {
        int mid = (low + high) / 2;

        // si queremos el minimo que cumple
        if (check(mid)) {
            ans = mid;
            high = mid - 1;
        }
    }
}
```

```
    } else {  
        low = mid + 1;  
    }  
  
    // Si quieres el maximo que cumple:  
    if (check(mid)) {  
        ans = mid;  
        low = mid + 1;  
    } else {  
        high = mid - 1;  
    }  
}  
  
cout << ans << endl;  
}
```

Listing 3.3: Búsqueda binaria en la respuesta

3.4.2. Ejemplo 1: Problema de Sumas

```
ll n;  
bool can(ll x, vll &a) {  
    ll cont = 0; // Sup. disponibles  
    fore(i, 0, n) {  
        if (x < a[i]) return false;  
        cont += x - a[i];  
    }  
    return cont >= x;  
}  
  
void solve() {  
    cin >> n;  
    vll a(n);  
    fore(i, 0, n) cin >> a[i];  
  
    ll l = 0, r = 1e14, ans = 0;  
    while (l <= r) {  
        ll mid = (l + r) >> 1;  
        if (can(mid, a)) {  
            ans = mid;  
            r = mid - 1;  
        } else {  
            l = mid + 1;  
        }  
    }  
    pri(ans);  
}
```

Listing 3.4: Problema de sumas con búsqueda binaria

3.4.3. Ejemplo 2: Problema de Porcentajes

```
ll k, x, y, a, b, n;  
vll pr;
```

```

bool can(ll st) {
    vll p(st);
    for (int i = a - 1; i < st; i += a) p[i] += x;
    for (int i = b - 1; i < st; i += b) p[i] += y;
    sort(all(p), greater<ll>());
    ll cur = 0;
    fore(i, 0, st) cur += p[i] * pr[i] / 100;
    return cur >= k;
}

void solve() {
    pr.clear();
    cin >> n;
    fore(i, 0, n) {
        ll ai;
        cin >> ai;
        pr.pb(ai);
    }
    sort(all(pr), greater<int>());
    cin >> x >> a >> y >> b >> k;

    ll l = 0, h = n, ans = -1;
    while (l <= h) {
        ll mid = l + (h - l) / 2;
        if (can(mid)) {
            ans = mid;
            h = mid - 1;
        } else {
            l = mid + 1;
        }
    }
    pri(ans);
}

```

Listing 3.5: Problema de porcentajes con tickets

3.4.4. Ejemplo 3: Problema de Recursos

```

ll k, n;
vi a, b;

bool can(ll mid) {
    ll ka = k;
    fore(i, 0, n) {
        ll totact = a[i] * mid;
        if (totact > b[i]) {
            ll rest = totact - b[i];
            if (rest <= ka) {
                ka -= rest;
            } else {
                return false;
            }
        }
    }
    return true;
}

```

```

void solve() {
    cin >> n >> k;
    a.resize(n);
    b.resize(n);
    for(i, 0, n) cin >> a[i];
    for(i, 0, n) cin >> b[i];

    ll low = 0, high = 2e9 + 10;
    ll res = -1;

    while (low <= high) {
        ll mid = (low + high) / 2;
        if (can(mid)) {
            res = mid;
            low = mid + 1; // Buscamos el MAXIMO
        } else {
            high = mid - 1;
        }
    }
    pri(res);
}

```

Listing 3.6: Problema de gestión de recursos

3.4.5. Consideraciones Importantes

- **Límites:** Definir correctamente l y r según el problema
- **Función can:** Debe ser eficiente (generalmente $O(n)$ o $O(n \log n)$)
- **Overflow:** Usar $\text{mid} = l + (r - l) / 2$ para evitar overflow
- **Tipo de búsqueda:** Decidir si se busca el mínimo o máximo que cumple

3.4.6. Complejidad

La complejidad total es $O(T \times \log(\text{rango}) \times \text{complejidad de can})$, donde:

- T : número de casos de prueba
- **rango:** diferencia entre r y l
- **can:** complejidad de la función de verificación

3.5. Comparadores personalizados

A veces necesitas ordenar de una forma específica, o hacer búsquedas binarias con criterios distintos al $<$ por defecto. Para eso existen los **comparadores**.

Un comparador es una función (o lambda) que recibe dos elementos y devuelve **true** si el primero **va antes** que el segundo en el orden deseado.

3.5.1. Comparadores en sort

Orden inverso con `greater<T>`

```
vector<int> v = {3, 1, 4, 1, 5, 9};
sort(all(v), greater<int>());
// v = {9, 5, 4, 3, 1, 1}
```

Listing 3.7: Ordenar de mayor a menor

Comparador con lambda

```
// Ordenar por segundo elemento del par, si empatan por primero
vector<pair<int,int>> v = {{1,3},{2,1},{3,2}};
sort(all(v), [](auto& a, auto& b){
    if (a.second != b.second) return a.second < b.second;
    return a.first < b.first;
});
// v = {{2,1},{3,2},{1,3}}
```

Listing 3.8: Ordenar con lambda personalizada

Comparador como struct (necesario para set y map)

Para `set` y `map`, las lambdas no sirven como comparador de tipo porque no tienen un tipo deducible en tiempo de compilación. Hay que usar un **struct con operator()**:

```
struct Cmp {
    bool operator()(const pair<int,int>& a, const pair<int,int>& b) const {
        // Ordena por suma, si empatan por primer elemento
        if (a.first + a.second != b.first + b.second)
            return a.first + a.second < b.first + b.second;
        return a.first < b.first;
    }
};

set<pair<int,int>, Cmp> s;
s.insert({2, 1}); // suma 3  <- primero
s.insert({0, 4}); // suma 4
s.insert({1, 3}); // suma 4, primer elem 1 <- segundo
```

Listing 3.9: Struct comparador para set

3.5.2. lower_bound y upper_bound con comparador

`lower_bound` y `upper_bound` aceptan un comparador como cuarto argumento. **Importante:** el vector debe estar ordenado con el mismo comparador.

```
vector<int> v = {9, 5, 4, 3, 1, 1};
// Vector ordenado de mayor a menor

// lower_bound con greater: primer elemento no mayor que val
// Es decir: primer elemento <= val
auto it = lower_bound(all(v), 4, greater<int>());
cout << *it << "\n"; // 4
```

```
// upper_bound con greater: primer elemento estrictamente menor que val
it = upper_bound(all(v), 4, greater<int>());
cout << *it << "\n"; // 3
```

Listing 3.10: lower_bound con greater (vector de mayor a menor)

Equivalencias útiles

Quiero encontrar...	Orden asc. (default)	Orden desc. (greater)
Primer elemento $\geq x$	lower_bound(..., x)	upper_bound(..., x, greater<T>())
Primer elemento $> x$	upper_bound(..., x)	lower_bound(..., x, greater<T>())
Último $\leq x$	prev(upper_bound(..., x))	lower_bound(..., x, greater<T>())
Último $< x$	prev(lower_bound(..., x))	upper_bound(..., x, greater<T>())

3.5.3. lower_bound en set y map

Para set y map **no usar la versión global** de lower_bound sino el **método del contenedor**: s.lower_bound(x). La versión global no conoce el comparador interno del set.

```
set<int> s = {1, 3, 5, 7, 9};

auto it = s.lower_bound(4); // primer elemento >= 4
cout << *it << "\n"; // 5

it = s.upper_bound(4);      // primer elemento > 4
cout << *it << "\n"; // 5

// Set en orden inverso
set<int, greater<int>> sr = {1, 3, 5, 7, 9};
// sr internamente: 9 7 5 3 1

// Con greater, lower_bound busca el primer elemento
// "no menor" segun greater, es decir: primer elemento <= val
auto it2 = sr.lower_bound(6);
cout << *it2 << "\n"; // 5

// upper_bound: primer elemento estrictamente "menor" segun greater,
// es decir: primer elemento < val
it2 = sr.upper_bound(6);
cout << *it2 << "\n"; // 5 (6 no estaba, mismo resultado)

it2 = sr.upper_bound(7);
cout << *it2 << "\n"; // 5 (primer elemento < 7 en orden desc)
```

Listing 3.11: lower_bound en set normal y set invertido

3.5.4. Comparadores para pares

Por defecto los pares se comparan lexicográficamente. Ejemplo de cómo modificarlo:

```
vector<pair<int, int>> v = {{3,1},{1,4},{2,2}};
```

```
// Ordenar por segundo elemento (ascendente)
sort(all(v), [](auto& a, auto& b){ return a.second < b.second; });
// v = {{3,1},{2,2},{1,4}}

// Ordenar por segundo elemento (descendente), si empatan
// por primer elemento descendente
sort(all(v), [](auto& a, auto& b){
    if (a.second != b.second) return a.second > b.second;
    return a.first > b.first;
});
// v = {{1,4},{2,2},{3,1}}
```

Listing 3.12: Ordenar pares por segundo elemento

3.5.5. Trampas comunes

- **Global lower_bound en set:** usar siempre `s.lower_bound(x)`, nunca `lower_bound(s.begin(), s.end(), x)` con un set con comparador custom.
- **Lambda en set:** las lambdas no se pueden usar como tipo de comparador en `set/map` directamente. Usar un struct con `operator()`.
- **Comparador inconsistente:** si ordenás con un comparador y después llamás a `lower_bound` con otro, el resultado es basura.
- **Strict weak ordering:** el comparador **debe** ser un orden estricto débil: irreflexivo, asimétrico y transitivo. `<=` como comparador **no** cumple esto y da comportamiento indefinido.

Capítulo 4

Matemática

4.1. Operaciones basicas

4.1.1. Obtener π

```
const double PI = 3.141592653589793; //FORMA 1
double pi = M_PI; //FORMA 2
```

Listing 4.1: Obtener π

4.1.2. Obtener la raiz cuadrada

Para calcular la raíz cuadrada de un número en C++, se utiliza la función `sqrt()`.

Ejemplo:

```
double x = 25.0;
double raiz = sqrt(x);
```

Notas importantes:

- El resultado puede guardarse en un `double` para conservar la precisión decimal.
- Si se guarda en un tipo entero, la parte decimal se truncará (se corta sin redondeo).

4.1.3. Máximo Común Divisor (MCD) y Mínimo Común Múltiplo (MCM)

El Máximo Común Divisor (MCD) de dos enteros a y b es el mayor entero que divide a ambos sin dejar residuo. El Mínimo Común Múltiplo (MCM) es el menor entero positivo que es divisible por ambos.

Relación fundamental

Existe una relación directa entre el MCD y el MCM de dos números a y b :

$$a \times b = \text{MCD}(a, b) \times \text{MCM}(a, b)$$

De aquí despejamos el MCM:

$$\text{MCM}(a, b) = \frac{a \times b}{\text{MCD}(a, b)}$$

Para evitar desbordamiento de enteros (overflow) al multiplicar $a \times b$, es una buena práctica realizar la división primero:

$$\text{MCM}(a, b) = \left(\frac{a}{\text{MCD}(a, b)} \right) \times b$$

Implementación en C++17 (Estándar)

Desde C++17, las funciones `std::gcd` y `std::lcm` están disponibles en la cabecera `<numeric>`:

```
#include <iostream>
#include <numeric> // Cabecera necesaria

using namespace std;

int main() {
    long long a = 12, b = 18;
    long long mcd = std::gcd(a, b); // Retorna 6
    long long mcm = std::lcm(a, b); // Retorna 36

    cout << "MCD: " << mcd << ", MCM: " << mcm << "\n";
    return 0;
}
```

Listing 4.2: MCD y MCM con la librería estándar de C++17

Implementación Manual (Algoritmo de Euclides)

Si estás usando una versión anterior a C++17 o necesitas programarlo a mano, puedes usar el algoritmo de Euclides de forma recursiva o iterativa:

```
// Complejidad: O(log(min(a, b)))
long long gcd(long long a, long long b) {
    return b == 0 ? a : gcd(b, a % b);
}

long long lcm(long long a, long long b) {
    if (a == 0 || b == 0) return 0;
    return (a / gcd(a, b)) * b;
}
```

Listing 4.3: Implementación manual de MCD y MCM

4.2. Reglas de Divisibilidad

Para determinar si un número es divisible por otro sin necesidad de hacer la división, se pueden usar las siguientes reglas de divisibilidad:

4.2.1. Reglas de Divisibilidad Básicas

1. **Divisible por 1:** Todos los números son divisibles por 1.
2. **Divisible por 2:** Un número es divisible por 2 si su último dígito es par (0, 2, 4, 6, 8).
3. **Divisible por 3:** Un número es divisible por 3 si la suma de sus dígitos es divisible por 3.
4. **Divisible por 4:** Un número es divisible por 4 si el número formado por sus últimos dos dígitos es divisible por 4.
5. **Divisible por 5:** Un número es divisible por 5 si su último dígito es 0 o 5.
6. **Divisible por 6:** Un número es divisible por 6 si es divisible por 2 y por 3.
7. **Divisible por 7:** Hay varias reglas:
 - **Regla 1:** Toma el último dígito, multiplícalo por 2 y réstalo del resto del número. Si el resultado es divisible por 7, entonces el número original también lo es.
 - **Regla 2:** Divide el número en bloques de 3 dígitos (de derecha a izquierda) y calcula la suma alternante de estos bloques. Si el resultado es divisible por 7, entonces el número original también lo es.
8. **Divisible por 8:** Un número es divisible por 8 si el número formado por sus últimos tres dígitos es divisible por 8.
9. **Divisible por 9:** Un número es divisible por 9 si la suma de sus dígitos es divisible por 9.
10. **Divisible por 10:** Un número es divisible por 10 si su último dígito es 0.

4.2.2. Reglas de Divisibilidad Menos Comunes

1. **Divisible por 11:**
 - **Regla 1:** Suma los dígitos en posiciones impares y resta la suma de los dígitos en posiciones pares. Si el resultado es divisible por 11, entonces el número original también lo es.
 - **Regla 2:** Alternativamente, puedes usar la suma alternante de los dígitos.
2. **Divisible por 12:** Un número es divisible por 12 si es divisible por 3 y por 4.
3. **Divisible por 13:**
 - **Regla 1:** Toma el último dígito, multiplícalo por 9 y réstalo del resto del número. Si el resultado es divisible por 13, entonces el número original también lo es.
 - **Regla 2:** Divide el número en bloques de 3 dígitos y calcula la suma alternante de estos bloques. Si el resultado es divisible por 13, entonces el número original también lo es.

4. Divisible por 17:

- **Regla 1:** Toma el último dígito, multiplícalo por 5 y réstalo del resto del número. Si el resultado es divisible por 17, entonces el número original también lo es.

5. Divisible por 19:

- **Regla 1:** Toma el último dígito, multiplícalo por 2 y súmalo al resto del número. Si el resultado es divisible por 19, entonces el número original también lo es.

4.2.3. Reglas de Divisibilidad para Números Primos Mayores

Para números primos mayores, como 23, 29, etc., las reglas pueden volverse más complejas y a menudo implican métodos similares a los usados para 7, 13, 17 y 19, pero con diferentes multiplicadores.

4.2.4. Métodos Generales para Verificar Divisibilidad

1. **División Directa:** La forma más directa de verificar la divisibilidad es realizar la división y ver si el residuo es cero.
2. **Uso de Factores:** Si un número es divisible por dos o más números coprimos, entonces es divisible por su producto. Por ejemplo, si un número es divisible por 3 y por 5, entonces es divisible por 15.

4.2.5. Generación de divisores

Para encontrar todos los divisores de un número k :

```
for (ll i = 1; i * i <= k; i++) {
    if (k % i == 0) {
        divs.push_back(i);
        if (i != k / i) divs.push_back(k / i);
    }
}
```

Listing 4.4: Generar los divisores de un numero

4.3. Números Primos

4.3.1. Chequear si un número es primo

Complejidad: $O(\sqrt{n})$

```
bool esPrimo(int n) {
    if (n <= 1) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; i * i <= n; i += 2)
        if (n % i == 0)
```

```

        return false;
        return true;
    }

```

Listing 4.5: Chequear si un número es primo

4.3.2. Criba de Eratóstenes para factorizar números

Uso

Guarda para cada número su **menor factor primo** (SPF, *smallest prime factor*). Con eso, puedes factorizar cualquier número $x \leq n$ muy rápido.

```

vector<int> spf; // spf[x] = menor factor primo de x

void build_spf(int n) {
    spf.assign(n + 1, 0);
    for (int i = 2; i <= n; i++) {
        if (spf[i] == 0) {
            spf[i] = i; // i es primo
            if (1LL * i * i <= n) {
                for (long long j = 1LL * i * i; j <= n; j += i) {
                    if (spf[j] == 0) spf[j] = i;
                }
            }
        }
    }
}

vector<pair<int,int>> factorize(int x) {
    vector<pair<int,int>> fac;
    while (x > 1) {
        int p = spf[x], e = 0;
        while (x % p == 0) {
            x /= p;
            e++;
        }
        fac.push_back({p, e});
    }
    return fac;
}

int main() {
    int n = 100; // limite maximo que vas a factorizar
    build_spf(n);

    int x = 6;
    auto f = factorize(x);

    // salida: 6 = 2^1 * 3^1
    cout << x << " = ";
    for (int i = 0; i < (int)f.size(); i++) {

```

```

        cout << f[i].first << "^" << f[i].second;
        if (i + 1 < (int)f.size()) cout << " * ";
    }
}

```

Listing 4.6: Criba SPF + factorización en primos

Complejidad:

- Preprocesamiento de la criba SPF hasta n : $O(n \log \log n)$.
- Memoria: $O(n)$.
- Factorizar un número x : $O(k)$, donde k es la cantidad de factores primos (con multiplicidad). En práctica: muy rápido.

4.4. Permutaciones

4.4.1. Computar las permutaciones

Sea un arreglo $A = [a_1, a_2, \dots, a_n]$. Queremos generar todas las permutaciones de sus elementos.

```

#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> A = {1, 2, 3};
    sort(A.begin(), A.end()); // ordenar antes de usar next_permutation

    do {
        for (int x : A) cout << x << " ";
        cout << "\n";
    } while (next_permutation(A.begin(), A.end()));

    return 0;
}

```

Listing 4.7: Chequear si un número es primo

4.5. Combinatoria

4.5.1. Triángulo de Pascal

El Triángulo de Pascal aparece muchísimo en programación competitiva porque permite calcular combinaciones binomiales de manera eficiente y simple.

Definición

Cada número es la suma de los dos que tiene arriba:

$$\begin{array}{ccccccc}
 & & & & 1 & & \\
 & & & & 1 & & 1 \\
 & & & 1 & & 2 & & 1 \\
 & & 1 & & 3 & & 3 & & 1 \\
 & 1 & & 4 & & 6 & & 4 & & 1 \\
 1 & & 5 & & 10 & & 10 & & 5 & & 1
 \end{array}$$

La fila n contiene los coeficientes binomiales:

$$\binom{n}{0}, \binom{n}{1}, \binom{n}{2}, \dots, \binom{n}{n}$$

Por ejemplo, la fila 5:

$$1 \ 5 \ 10 \ 10 \ 5 \ 1$$

corresponde a:

$$\binom{5}{0}, \binom{5}{1}, \binom{5}{2}, \binom{5}{3}, \binom{5}{4}, \binom{5}{5}$$

Relación fundamental

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

Esta ecuación es exactamente la regla del triángulo de Pascal.

Construcción en $O(n^2)$

```

vector<vector<long long>> C(n + 1, vector<long long>(n + 1));

for (int i = 0; i <= n; i++) {
    C[i][0] = C[i][i] = 1;

    for (int j = 1; j < i; j++) {
        C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
    }
}

```

Listing 4.8: Construcción del triángulo de Pascal

Luego, para realizar consultas como $\binom{5}{2}$:

```
cout << C[5][2]; // imprime 10
```

Listing 4.9: Ejemplo de consulta

Propiedades útiles

- **Suma de una fila:**

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

Ejemplo: la fila 1 4 6 4 1 suma $16 = 2^4$.

- **Coefficientes del binomio:** El binomio $(a + b)^n$ tiene coeficientes dados por la fila n . Por ejemplo:

$$(a + b)^4 = a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4$$

Los coeficientes 1 4 6 4 1 provienen de la fila 4 de Pascal.

4.6. Geometría

4.6.1. Distancia entre dos puntos

Dados dos puntos (x_1, y_1) y (x_2, y_2) , la distancia euclidiana es:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

En programación competitiva, muchas veces conviene usar la distancia al cuadrado (para evitar `sqrt`):

$$d^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

```
11 dist2 = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
```

Listing 4.10: Distancia al cuadrado entre dos puntos

4.6.2. Triángulos

Un triángulo queda determinado si conocés:

- 3 lados (SSS)
- 2 lados + 1 ángulo (SAS)
- 2 ángulos + 1 lado (ASA / AAS)

Ley de Cosenos

Se usa cuando tenés lados y querés ángulos, o viceversa:

$$c^2 = a^2 + b^2 - 2ab \cos(C)$$

Sirve para:

- SSS $\rightarrow$ sacar ángulos
- SAS $\rightarrow$ sacar el lado faltante

Ejemplo (CP):

```
double C = acos((a * a + b * b - c * c) / (2.0 * a * b));
```

Listing 4.11: Ley de Cosenos: obtener un ángulo

Ley de Senos

Cuando tenés ángulos y lados mezclados:

$$\frac{\sin(A)}{a} = \frac{\sin(B)}{b} = \frac{\sin(C)}{c}$$

Sirve para:

- ASA / AAS → sacar lados
- completar triángulos

Ejemplo:

```
double b = a * sin(B) / sin(A);
```

Listing 4.12: Ley de Senos: obtener un lado

4.7. Transformada Rápida de Fourier (FFT)

La FFT permite multiplicar dos polinomios de grado n en $O(n \log n)$, mucho mejor que la multiplicación ingenua $O(n^2)$. También se usa para multiplicar números grandes y para **convolución** de secuencias.

4.7.1. Idea general

Dados dos polinomios A y B , queremos calcular $C = A \cdot B$. El truco es:

1. Evaluar A y B en n puntos especiales (las raíces n -ésimas de la unidad): $O(n \log n)$
2. Multiplicar los valores punto a punto: $O(n)$
3. Recuperar los coeficientes de C con la DFT inversa: $O(n \log n)$

4.7.2. Transformada Discreta de Fourier (DFT)

Sea $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ un polinomio de grado $n-1$, donde n es potencia de 2.

Las **raíces n -ésimas de la unidad** son los n números complejos:

$$w_n^k = e^{\frac{2\pi i k}{n}} = \cos\left(\frac{2\pi k}{n}\right) + i \sin\left(\frac{2\pi k}{n}\right), \quad k = 0, 1, \dots, n-1$$

La **DFT** del vector de coeficientes $(a_0, \dots, a_{n-1})$ es el vector de valores:

$$y_k = A(w_n^k) = \sum_{j=0}^{n-1} a_j w_n^{jk}, \quad k = 0, \dots, n-1$$

La **DFT inversa** recupera los coeficientes a partir de los valores:

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k w_n^{-jk}$$

Es decir, la DFT inversa es igual a la DFT pero usando w_n^{-1} y dividiendo por n al final.

4.7.3. Multiplicación de polinomios con DFT

Sean dos polinomios A y B . Para calcular $C = A \cdot B$:

$$C = \text{DFT}^{-1}(\text{DFT}(A) \cdot \text{DFT}(B))$$

donde el producto del medio es elemento a elemento. Esto funciona porque evaluar en los mismos puntos y multiplicar los valores equivale a multiplicar los polinomios.

4.7.4. Algoritmo: FFT recursiva (Cooley-Tukey)

La clave del algoritmo es dividir el polinomio en partes par e impar:

$$A(x) = A_{\text{par}}(x^2) + x \cdot A_{\text{impar}}(x^2)$$

donde A_{par} tiene los coeficientes de índice par y A_{impar} los de índice impar. Se aplica recursivamente, reduciendo el problema de tamaño n a dos de tamaño $n/2$.

```
using cd = complex<double>;
const double PI = acos(-1.0);

// fft(a, false) -> DFT directa
// fft(a, true) -> DFT inversa
void fft(vector<cd>& a, bool inv) {
    int n = a.size();
    if (n == 1) return;

    // Separar coeficientes pares e impares
    vector<cd> a0(n / 2), a1(n / 2);
    for (int i = 0; i < n / 2; i++) {
        a0[i] = a[2 * i];
        a1[i] = a[2 * i + 1];
    }

    // Llamadas recursivas
    fft(a0, inv);
    fft(a1, inv);

    // Raiz primitiva n-esima de la unidad
    double ang = 2 * PI / n * (inv ? -1 : 1);
    cd w(1), wn(cos(ang), sin(ang));

    for (int i = 0; i < n / 2; i++) {
        a[i] = a0[i] + w * a1[i];
        a[i + n / 2] = a0[i] - w * a1[i];
        if (inv) {
            a[i] /= 2;
        }
    }
}
```

```

        a[i + n / 2] /= 2;
    }
    w *= wn;
}
}

```

Listing 4.13: FFT recursiva de Cooley-Tukey

Complejidad: $O(n \log n)$ tiempo y $O(n)$ espacio extra (por la recursión).

4.7.5. Función multiply: convolución de polinomios

Dados los vectores de coeficientes de A y B , devuelve los coeficientes del producto $C = A \cdot B$.

```

// Recibe coeficientes de A y B (indice = grado del termino)
// Devuelve coeficientes de C = A * B
vector<ll> multiply(vector<ll>& fa, vector<ll>& fb) {
    vector<cd> a(fa.begin(), fa.end());
    vector<cd> b(fb.begin(), fb.end());

    // n debe ser potencia de 2 y mayor que deg(A) + deg(B)
    int n = 1;
    while (n < (int)(fa.size() + fb.size()))
        n <<= 1;
    a.resize(n);
    b.resize(n);

    // DFT directa de A y B
    fft(a, false);
    fft(b, false);

    // Multiplicacion punto a punto
    for (int i = 0; i < n; i++)
        a[i] *= b[i];

    // DFT inversa
    fft(a, true);

    // Pasar a enteros (redondear para evitar error de punto flotante)
    vector<ll> res(n);
    for (int i = 0; i < n; i++)
        res[i] = llround(a[i].real());

    return res;
}

```

Listing 4.14: Multiplicacion de polinomios con FFT

Ejemplo de uso

```

// A(x) = 1 + 2x + 3x^2 -> coeficientes: {1, 2, 3}
// B(x) = 4 + 5x         -> coeficientes: {4, 5}
// C(x) = 4 + 13x + 22x^2 + 15x^3

vector<ll> a = {1, 2, 3};
vector<ll> b = {4, 5};

```

```

auto c = multiply(a, b);
// c = {4, 13, 22, 15, 0, 0, 0, 0}
// (los ceros del final son padding, ignorarlos)

```

Listing 4.15: Multiplicar $(1 + 2x + 3x^2) \cdot (4 + 5x)$

4.7.6. Convolución para contar coincidencias

Un uso muy frecuente en competencia: dado un patrón y un texto, contar cuántos lugares coinciden. Se modela como convolución de polinomios donde los coeficientes son 0 o 1.

```

// Dado un arreglo, contar pares (i,j) con a[i] + a[j] = k
// 1. Construir vector de frecuencias f donde f[v] = cant de veces que aparece v
// 2. C = multiply(f, f)
// 3. C[k] = cantidad de pares (con repeticion e i=j incluidos)

int MAXV = 1e6 + 1;
vector<ll> f(MAXV, 0);
for (int x : a) f[x]++;

auto c = multiply(f, f);
// c[k] = cantidad de pares (a[i], a[j]) con a[i] + a[j] = k
// Recordar descontar pares (x, x) si no se permiten i = j

```

Listing 4.16: Contar sumas de pares con FFT (frecuencias)

4.7.7. Consideraciones importantes

- **Potencia de 2:** el tamaño del vector pasado a `fft` debe ser siempre potencia de 2. Si no lo es, hay que hacer *padding* con ceros.
- **Tamaño del resultado:** el producto de polinomios de grados d_A y d_B tiene grado $d_A + d_B$, por lo tanto el vector de la FFT debe tener tamaño $\geq d_A + d_B + 1$ (redondeado a potencia de 2).
- **Precisión:** la FFT con `double` tiene errores de punto flotante. Para coeficientes grandes (o polinomios largos), usar `llround` al convertir el resultado final. Si los coeficientes son muy grandes, puede ser necesaria la **FFT de número teórico** (NTT) con aritmética modular.
- **Versión iterativa:** la versión recursiva es simple pero tiene overhead por las llamadas. Para tiempo crítico existe una versión iterativa con reordenamiento bit-reversal que es más rápida en la práctica.

4.7.8. Complejidad

- **FFT (DFT directa o inversa):** $O(n \log n)$
- **Multiplicación de polinomios:** $O(n \log n)$ donde $n = d_A + d_B$
- **Multiplicación trivial** (para comparar): $O(n^2)$

Capítulo 5

Programacion Dinamica (dp)

5.1. De una función recursiva por partes a código con DP

En muchos problemas, partimos de una definición de función recursiva por partes. El objetivo es transformarla en un algoritmo eficiente usando programación dinámica. A continuación se muestra un ejemplo general del proceso.

Ejemplo: función definida por partes

Supongamos que tenemos la siguiente definición matemática:

$$f(i) = \begin{cases} 0 & \text{si } i = 0 \\ f(i-1) + |a[i] - a[i-1]| & \text{si } i = 1 \\ \min(f(i-1) + |a[i] - a[i-1]|, f(i-2) + |a[i] - a[i-2]|) & \text{si } i \geq 2 \end{cases}$$

Queremos implementarla eficientemente para valores grandes de i .

Paso 1: Identificar el estado

El subproblema está determinado solo por el índice i . Por lo tanto, usamos un arreglo $dp[i]$ donde $dp[i]$ almacena el valor de $f(i)$.

Paso 2: Código en C++ con memoización

```
const int MAXN = 1e5+10;
int dp[MAXN];
int a[MAXN];

int f(int i) {
    if (dp[i] != -1) return dp[i];

    if (i == 0) return dp[i] = 0;
    if (i == 1) return dp[i] = f(0) + abs(a[1] - a[0]);
```

```

return dp[i] = min(
f(i - 1) + abs(a[i] - a[i - 1]),
f(i - 2) + abs(a[i] - a[i - 2])
);
}

```

Listing 5.1: Función recursiva con memoización

Inicialización:

```
fill(dp, dp + n, -1);
```

Llamada inicial:

```
cout << f(n - 1) << '\n';
```

Paso 3: Versión iterativa (tabulación)

```

dp[0] = 0;
dp[1] = abs(a[1] - a[0]);

for (int i = 2; i < n; ++i) {
dp[i] = min(
dp[i - 1] + abs(a[i] - a[i - 1]),
dp[i - 2] + abs(a[i] - a[i - 2])
);
}

```

Listing 5.2: Versión iterativa (bottom-up)

Resumen

- La clave está en identificar bien los **casos base**.
- Luego traducir cada parte de la definición en código con condiciones explícitas.
- Asegurate de guardar los resultados para no recalcular.

5.2. Ejemplos clásicos

5.2.1. A - Frog 1

Problema: dado un arreglo de alturas, calcular el costo mínimo para saltar desde el índice 0 al último, con saltos de 1 o 2 posiciones.

```

11 dp[MAXN];
11 a[MAXN];
11 f(int i) {
11 if (i == 0) return 0;
11 if (dp[i] != -1) return dp[i];
11 res = f(i-1) + abs(a[i] - a[i-1]);
11 if (i > 1) res = min(res, f(i-2) + abs(a[i] - a[i-2]));

```

```
return dp[i] = res;
}
```

Listing 5.3: Frog1: DP mínima energía

Estados: posición actual. Transiciones: desde ‘i-1’ o ‘i-2’. Caso base: ‘dp[0] = 0’.

5.2.2. C - Vacation

Problema: en cada día, podés hacer una de 3 actividades con recompensa. No podés repetir la misma actividad dos días seguidos.

```
11 dp[MAXN][3];
11 a[MAXN][3];

11 f(int i, int last) {
11     if (i == n) return 0;
11     if (dp[i][last] != -1) return dp[i][last];
11     res = 0;
11     for(j, 0, 3) {
11         if (j != last) {
11             res = max(res, a[i][j] + f(i+1, j));
11         }
11     }
11     return dp[i][last] = res;
11 }
```

Listing 5.4: Vacation: DP sobre días y última actividad

Estado: día actual y actividad anterior. Transiciones: elegir una actividad distinta a la del día anterior. Caso base: cuando ‘i == n’.

5.2.3. D - Knapsack 0/1

Problema clásico: elegir un subconjunto de objetos sin pasar un peso máximo.

```
11 dp[MAXW];
11 int w[MAXN], v[MAXN];

11 void solve() {
11     for (int i = 0; i < n; ++i) {
11         for (int j = W; j >= w[i]; --j) {
11             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
11         }
11     }
11 }
```

Listing 5.5: Mochila 0/1: DP por peso

Estado: peso total actual. Transiciones: elegir o no cada objeto. Usa optimización de espacio (1D).

5.2.4. G - Longest Path in DAG

Problema: encontrar el camino más largo en un grafo dirigido acíclico.

```
vector<int> G[MAXN];
int dp[MAXN];

int dfs(int u) {
    if (dp[u] != -1) return dp[u];
    int res = 0;
    for (int v : G[u]) {
        res = max(res, 1 + dfs(v));
    }
    return dp[u] = res;
}
```

Listing 5.6: Camino más largo en DAG con DP + DFS

Estado: nodo actual. Transiciones: recorrer vecinos. Requiere topological sort o memoización con DFS.

5.3. Resumen

- Pensar en el subproblema mínimo que necesitás.
- Formular la recurrencia primero.
- Escribir la versión recursiva con memoización es buena forma de empezar.
- Luego podés optimizar a iterativa si hace falta.

Capítulo 6

Grafos

6.1. Representaciones comunes

Lista de adyacencia

```
vector<int> G[MAXN]; // grafo sin peso

void agregar_arista(int u, int v) {
    G[u].push_back(v);
    G[v].push_back(u); // si es no dirigido
}
```

Listing 6.1: Lista de adyacencia

Matriz de adyacencia

```
bool mat[MAXN][MAXN];

void agregar_arista(int u, int v) {
    mat[u][v] = true;
    mat[v][u] = true; // si es no dirigido
}
```

Listing 6.2: Matriz de adyacencia

6.2. Recorridos clásicos

6.2.1. DFS (Depth-First Search)

```
bool vis[MAXN];
vector<int> G[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) dfs(v);
    }
}
```

```

    }
}

```

Listing 6.3: DFS no dirigido

6.2.2. BFS (Breadth-First Search)

BFS recorre el grafo por capas (nivel por nivel), lo que garantiza que la primera vez que llega a un nodo, lo hace por el **camino más corto** (en cantidad de aristas).

BFS básico con distancias

```

int dist[MAXN];
bool vis[MAXN];
vector<int> G[MAXN];

void bfs(int s) {
    queue<int> q;
    q.push(s);
    vis[s] = true;
    dist[s] = 0;

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (!vis[v]) {
                vis[v] = true;
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

```

Listing 6.4: BFS con calculo de distancias

Despues de llamar `bfs(s)`, `dist[v]` contiene la distancia minima (en cantidad de aristas) desde `s` hasta `v`. Si `vis[v] == false`, el nodo `v` no es alcanzable desde `s`.

Obtener la distancia mínima entre dos nodos

Para saber la distancia entre `s` y un nodo destino `t`:

```

bfs(s);

if (!vis[t]) {
    // t no es alcanzable desde s
} else {
    cout << dist[t] << "\n"; // distancia minima s -> t
}

```

Listing 6.5: Consultar distancia minima

Reconstruir el camino mínimo

Para recuperar **la secuencia de nodos** del camino mas corto, se agrega un arreglo `parent[]` que registra, para cada nodo, desde cual nodo fue visitado por primera vez.

BFS con arreglo parent:

```
int dist[MAXN], par[MAXN];
bool vis[MAXN];
vector<int> G[MAXN];

void bfs(int s, int n) {
    fill(vis, vis + n, false);
    fill(par, par + n, -1);

    queue<int> q;
    vis[s] = true;
    dist[s] = 0;
    q.push(s);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : G[u]) {
            if (!vis[v]) {
                vis[v] = true;
                dist[v] = dist[u] + 1;
                par[v] = u; // u es el padre de v
                q.push(v);
            }
        }
    }
}
```

Listing 6.6: BFS con reconstruccion de camino

Reconstruir el camino desde s hasta t:

Una vez que termina el BFS, el camino esta encadenado hacia atras a traves de `par[]`. Se sigue desde `t` hasta llegar a `-1` (el origen), y luego se invierte.

```
vector<int> getPath(int s, int t) {
    if (!vis[t]) return {}; // t no alcanzable

    vector<int> path;
    for (int v = t; v != -1; v = par[v])
        path.push_back(v);

    reverse(path.begin(), path.end());
    // path = {s, ..., t}
    return path;
}
```

Listing 6.7: Funcion para reconstruir el camino

Ejemplo completo:

```

bfs(0, n);

auto path = getPath(0, 4);

cout << "Distancia: " << dist[4] << "\n";
cout << "Camino: ";
for (int v : path) cout << v << " ";
cout << "\n";
// Salida: Distancia: 3
//           Camino: 0 2 3 4

```

Listing 6.8: Ejemplo s

- **Distancia:** $\text{dist}[t]$ despues de correr $\text{bfs}(s)$.
- **Camino:** seguir $\text{par}[t] \rightarrow \text{par}[\text{par}[t]] \rightarrow \dots$ hasta llegar a -1 , luego invertir.
- **Complejidad:** $O(n + m)$ para construccion del BFS.

6.3. Componentes conexas

Para contar la cantidad de componentes conexas en un grafo no dirigido y guardar un nodo representativo (representante) de cada una de ellas, podemos usar el siguiente enfoque con DFS:

```

const ll MAXN = 1e5 + 10;
bool vis[MAXN];
vector<ll> g[MAXN];
vector<ll> lista; // Guarda un nodo representativo de cada componente

void dfs(ll u) {
    vis[u] = true;
    for (ll v : g[u]) {
        if (!vis[v]) {
            dfs(v);
        }
    }
}

ll comp(ll n) {
    ll ans = 0;
    lista.clear();
    for(i, 0, n) {
        if (!vis[i]) {
            dfs(i);
            ans++;
            lista.pb(i + 1); // Guarda el nodo representativo (1-indexed)
        }
    }
    return ans;
}

```

Listing 6.9: Contar componentes conexas y guardar representantes

Detalles de uso

- **Resultado:** Al llamar a `comp(n)`, la función devuelve la ****cantidad total de componentes conexas**** del grafo.
- **Representantes:** El vector `lista` contendrá un nodo representativo por cada componente hallada (en este caso guardado con indexación `1, i + 1`).
- **Complejidad:** $\mathcal{O}(n + m)$ ya que se visita cada nodo y cada arista una sola vez.

6.4. Dijkstra (Caminos Mínimos con Pesos Positivos)

El algoritmo de Dijkstra permite encontrar la distancia mínima desde un nodo origen s a todos los demás nodos en un grafo con **pesos no negativos**. Utiliza una cola de prioridad (`priority_queue`) para procesar siempre el nodo con la menor distancia acumulada hasta el momento.

Representación y Lectura de un Grafo con Pesos

Para almacenar un grafo con pesos, la lista de adyacencia guarda pares de la forma `{vecino, peso}`:

```
const ll MAXN = 1e5 + 10;
// g[u] contiene pares {v, w} donde v es el destino y w el peso de la arista
vector<pair<ll, ll>> g[MAXN];

void agregar_arista(ll u, ll v, ll w, bool dirigido = false) {
    g[u].push_back({v, w});
    if (!dirigido) {
        g[v].push_back({u, w}); // Si el grafo es no dirigido
    }
}
```

Listing 6.10: Representación de grafo ponderado

Para leer las aristas desde la entrada estándar (1-indexed):

```
ll n, m; cin >> n >> m;
for(i, 0, m) {
    ll u, v, w;
    cin >> u >> v >> w;
    g[u].push_back({v, w});
    // g[v].push_back({u, w}); // Descomentar si es no dirigido
}
```

Listing 6.11: Lectura de aristas ponderadas

Implementación de Dijkstra

La `priority_queue` en C++ ordena los elementos de mayor a menor por defecto. Para que funcione como una *min-heap* (menor distancia primero), se utiliza `greater<pair<ll, ll>>`.

```
const ll INF = 1e18 + 10;
const ll MAXN = 1e5 + 10;

vector<pair<ll, ll>> g[MAXN]; // {destino, peso}
ll dist[MAXN];
ll par[MAXN]; // Para reconstruccion del camino (opcional)

void dijkstra(ll s, ll n) {
    // priority_queue guarda pares {distancia_acumulada, nodo}
    priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>>
    pq;

    fill(dist, dist + n + 1, INF);
    fill(par, par + n + 1, -1);

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        // Si encontramos una distancia guardada menor, ignoramos esta entrada
        if (d > dist[u]) continue;

        for (auto [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                par[v] = u; // Guardar el padre para reconstruir el camino
                pq.push({dist[v], v});
            }
        }
    }
}
```

Listing 6.12: Algoritmo de Dijkstra con C++

Reconstrucción del Camino Mínimo

Para obtener la secuencia exacta de nodos del camino mínimo desde s hasta t :

```
vector<ll> obtenerCamino(ll s, ll t) {
    if (dist[t] == INF) return {}; // t no es alcanzable desde s

    vector<ll> camino;
    for (ll v = t; v != -1; v = par[v]) {
        camino.push_back(v);
    }
    reverse(camino.begin(), camino.end());
}
```

```

    return camino;
}

```

Listing 6.13: Recuperar el camino desde origen s a destino t

Detalles de uso y Complejidad

- **Complejidad temporal:** $\mathcal{O}((N + M) \log N)$, adecuada para $N, M \leq 10^5$ o $2 \cdot 10^5$.
- **Tipo de datos:** Se utiliza `ll` (`long long`) y `INF = 1e18` para prevenir desbordamientos (*overflow*) durante las sumas de pesos.
- **Restricción:** Solo funciona si todos los pesos $w \geq 0$. Si existen pesos negativos, usar Bellman-Ford o SPFA.

6.5. Bellman-Ford

```

vector<tuple<int, int, int>> edges; // {u, v, peso}
int dist[MAXN];

bool bellman(int s, int n) {
    fill(dist, dist + MAXN, INF);
    dist[s] = 0;

    for (int i = 0; i < n - 1; ++i) {
        for (auto [u, v, w] : edges) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    // Detectar ciclos negativos
    for (auto [u, v, w] : edges) {
        if (dist[u] + w < dist[v]) return true;
    }

    return false;
}

```

Listing 6.14: Bellman-Ford para grafos con pesos negativos

6.6. Problemas clásicos

A - Número de componentes conexas

Estado: nodos visitados. **Recorrido:** DFS o BFS. **Complejidad:** $\mathcal{O}(n + m)$

B - Camino más corto (no ponderado)

Idea: usar BFS desde el nodo origen y guardar las distancias.

C - Dijkstra / Bellman-Ford

Cuando usar cada uno:

- Dijkstra → aristas no negativas. - Bellman-Ford → permite pesos negativos.

6.7. ¿Cómo verificar si un grafo es una línea (path)?

1. Verificar que $M = N - 1$ (condición necesaria para ser un árbol).
2. Verificar que el grafo es conexo (DFS o BFS desde cualquier nodo).
3. Contar los grados de los nodos:
 - Debe haber exactamente 2 nodos con grado 1.
 - Todos los demás deben tener grado 2.

Código en C++

```
vector<int> G[MAXN];
bool vis[MAXN];
int deg[MAXN];

void dfs(int u) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) dfs(v);
    }
}

bool es_linea(int n, int m) {
    if (m != n - 1) return false; // debe tener N-1 aristas

    // verificar conectividad
    fill(vis, vis + n, false);
    dfs(0);
    for (int i = 0; i < n; ++i)
        if (!vis[i]) return false;

    // contar grados
    int cnt_deg1 = 0;
    for (int i = 0; i < n; ++i) {
        if (deg[i] == 1) cnt_deg1++;
        else if (deg[i] != 2) return false;
    }

    return cnt_deg1 == 2;
}
```

```
}

```

Listing 6.15: Verificar si un grafo es una línea

Entrada:

- Llenar $G[i]$ con las aristas del grafo.
- Aumentar $\deg[u]$ y $\deg[v]$ cada vez que se agrega una arista entre u y v .

6.8. Detección de Ciclos

6.8.1. Grafo No Dirigido

En un grafo no dirigido, un ciclo existe si durante un recorrido DFS alcanzamos un nodo ya visitado que **no es el padre** directo del nodo actual.

Algoritmo y Reconstrucción

Para recuperar los nodos que forman el ciclo:

1. Se guardan los extremos de la arista que cierra el ciclo en las variables `cicloU` y `cicloV`.
2. Se reconstruye el camino desde `cicloU` hacia atrás usando `par[]` hasta llegar a `cicloV`.
3. Se añade `cicloV` al final para cerrar el ciclo explícitamente.

```
const ll MAXN = 1e5 + 10;
bool vis[MAXN];
ll par[MAXN];
vector<ll> g[MAXN];
ll cicloU = -1, cicloV = -1;

bool dfs(ll u, ll parent) {
    vis[u] = true;
    par[u] = parent;

    for (ll v : g[u]) {
        if (!vis[v]) {
            if (dfs(v, u)) return true;
        } else if (v != parent) {
            // Se encontró una arista de retroceso que cierra el ciclo
            cicloU = u;
            cicloV = v;
            return true;
        }
    }
    return false;
}
```

```

vector<ll> obtenerCiclo() {
    vector<ll> ciclo;
    // Reconstruimos el camino desde cicloU hasta cicloV a traves de los padre
    for (ll curr = cicloU; curr != cicloV; curr = par[curr]) {
        ciclo.push_back(curr);
    }
    ciclo.push_back(cicloV);
    ciclo.push_back(cicloU); // Cerrar el ciclo (opcional)
    return ciclo;
}

```

Listing 6.16: Detectar y reconstruir ciclo en grafo no dirigido (1-indexed)

Cómo usarlo

```

int main() {
    // Lectura e inicializacion de variables...
    fill(vis, vis + n + 1, false);
    fill(par, par + n + 1, -1);
    cicloU = -1; cicloV = -1;

    bool hay_ciclo = false;
    fore(i, 1, n + 1) { // Iteracion 1-indexed para todas las componentes
        if (!vis[i]) {
            if (dfs(i, -1)) {
                hay_ciclo = true;
                break;
            }
        }
    }

    if (hay_ciclo) {
        vector<ll> ciclo = obtenerCiclo();
        cout << "Ciclo detectado de tamaño " << ciclo.size() - 1 << ":\n";
        for (ll nodo : ciclo) cout << nodo << " ";
        cout << "\n";
    } else {
        cout << "IMPOSSIBLE\n";
    }
}

```

Listing 6.17: Uso en la funcion principal

6.8.2. Grafo Dirigido

En un grafo dirigido se utiliza el algoritmo de colores para identificar **aristas de retroceso** (*back edges*), las cuales apuntan a nodos que aún se encuentran en la pila de recursión activa.

- **0 (Blanco):** Nodo no visitado.

- **1 (Gris):** Nodo en proceso de exploración (en la pila de recursión).
- **2 (Negro):** Nodo completamente visitado.

```
int color[MAXN]; // 0: blanco, 1: gris, 2: negro
vector<ll> g[MAXN];

bool dfsDirigido(ll u) {
    color[u] = 1; // En proceso (gris)

    for (ll v : g[u]) {
        if (color[v] == 0) {
            if (dfsDirigido(v)) return true;
        } else if (color[v] == 1) {
            return true; // Encontrado nodo en la pila -> Hay ciclo
        }
    }

    color[u] = 2; // Procesado (negro)
    return false;
}

bool hayCicloDirigido(ll n) {
    fill(color, color + n + 1, 0);
    for(i, 1, n + 1) {
        if (color[i] == 0 && dfsDirigido(i))
            return true;
    }
    return false;
}
```

Listing 6.18: Detectar ciclo en grafo dirigido

6.9. Verificar si hay ciclo impar

1. Intentar colorear el grafo con 2 colores (bipartición)
2. Si no es posible, contiene ciclo impar

Código en C++

```
vector<int> G[MAXN];
int col[MAXN]; // 0 o 1

bool tieneCicloImpar(int n) {
    queue<int> q;
    fill(col, col + n, -1);

    col[0] = 0;
    q.push(0);
```

```

while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : G[u]) {
        if (col[v] == -1) {
            col[v] = 1 - col[u];
            q.push(v);
        }
        else if (col[v] == col[u]) {
            return true; // Ciclo impar encontrado
        }
    }
}
return false;
}

```

Listing 6.19: Detectar ciclo impar

6.10. Verificar si el grafo es bipartito (o tiene ciclo impar)

Un grafo es **bipartito** si sus vértices se pueden dividir en dos conjuntos tales que no existan aristas entre vértices del mismo conjunto. Esto es equivalente a decir que el grafo ****no contiene ciclos impares****.

Algoritmo

1. Se intenta colorear el grafo con 2 colores (0 y 1) usando BFS.
2. Si al recorrer un vecino este ya fue coloreado y tiene el mismo color que el nodo actual ($\text{col}[v] == \text{col}[u]$), significa que existe un ciclo impar y el grafo **no es bipartito**.
3. Se debe iterar sobre todos los nodos del grafo para asegurar la verificación en ****todas las componentes conexas****.

Código en C++

```

const ll MAXN = 1e5 + 10;
ll col[MAXN];
vector<ll> g[MAXN];

bool bfs(ll s) {
    queue<ll> q;
    col[s] = 0;
    q.push(s);

    while (!q.empty()) {
        ll u = q.front();

```

```

        q.pop();
        for (ll v : g[u]) {
            if (col[v] == -1) {
                col[v] = 1 - col[u]; // Color opuesto
                q.push(v);
            } else if (col[v] == col[u]) {
                return false; // Conflicto de colores (ciclo impar)
            }
        }
    }
    return true;
}

bool esBipar(ll n) {
    fill(col, col + n + 1, -1);
    fore(i, 1, n + 1) { // Iterar sobre todas las componentes (1-indexed)
        if (col[i] == -1) {
            if (!bfs(i)) return false;
        }
    }
    return true;
}

```

Listing 6.20: Verificar si un grafo es bipartito (1-indexed)

Detalles de uso

- **Indexación:** El código asume nodos numerados de 1 a n . Si es 0-indexed, ajustar la iteración en `esBipar` a `fore(i, 0, n)`.
- **Complejidad:** $\mathcal{O}(n + m)$ ya que procesa cada nodo y arista una sola vez.

6.11. Verificar si el grafo es un árbol

1. Verificar que tiene exactamente $N-1$ aristas
2. Verificar que es conexo
3. Verificar que no tiene ciclos

Código en C++

```

vector<int> G[MAXN];
bool vis[MAXN];

bool hasCycle(int u, int parent) {
    vis[u] = true;
    for (int v : G[u]) {
        if (!vis[v]) {
            if (hasCycle(v, u)) return true;
        }
    }
    return false;
}

```

```

        }
        else if (v != parent) return true;
    }
    return false;
}

bool esArbol(int n, int m) {
    if (m != n - 1) return false;

    // Verificar conexidad
    fill(vis, vis + n, false);
    hasCycle(0, -1);
    for (int i = 0; i < n; ++i)
        if (!vis[i]) return false;

    // Verificar que no tiene ciclos
    fill(vis, vis + n, false);
    if (hasCycle(0, -1)) return false;

    return true;
}

```

Listing 6.21: Verificar si grafo es árbol

6.12. Operaciones básicas en árboles

Obtener el padre de cada nodo

Durante un DFS, el nodo desde el que llegaste es el padre.

```

vector<vector<ll>> arb(n);
vector<ll> padre(n, -1);

void dfs(ll u, ll p){
    padre[u] = p;

    for(ll v : arb[u]){
        if(v == p) continue;
        dfs(v, u);
    }
}

```

Listing 6.22: Obtener padre de cada nodo

Llamás:

```
dfs(0, -1);
```

Listing 6.23: Llamada inicial del DFS

Obtener la cantidad de hijos

Cada vez que desde un nodo visitás un vecino que no es su padre, encontraste un hijo.

```
vector<ll> hijos(n, 0);

void dfs(ll u, ll p){
    padre[u] = p;

    for(ll v : arb[u]){
        if(v == p) continue;

        hijos[u]++;

        dfs(v, u);
    }
}
```

Listing 6.24: Obtener cantidad de hijos

Obtener el grado de cada nodo

El grado no depende de la raíz. Es simplemente la cantidad de vecinos.
Como estás usando listas de adyacencia:

```
vector<ll> grado(n);

for(ll i = 0; i < n; i++)
    grado[i] = arb[i].size();
```

Listing 6.25: Obtener grado de un nodo

O directamente imprimiendo:

```
cout << arb[i].size();
```

Listing 6.26: Imprimir grado

6.13. Número de hojas en un árbol

1. Contar nodos con grado 1 (excepto posiblemente la raíz en árboles enraizados)

Código en C++

```
vector<int> G[MAXN];
int deg[MAXN];

int contarHojas(int n) {
    int hojas = 0;
    for (int i = 0; i < n; ++i) {
        if (G[i].size() == 1) // grado 1
            hojas++;
    }
    return hojas;
}
```

```
}
```

Listing 6.27: Contar hojas en un árbol

6.14. Diámetro de un árbol

El algoritmo más usado en programación competitiva para obtener el diámetro de un árbol es hacer dos DFS (o dos BFS).

Idea

El diámetro es el camino simple más largo del árbol. En un árbol, si empezás desde cualquier nodo y buscás el más lejano, siempre terminás en uno de los extremos del diámetro. Luego, al volver a buscar desde ese extremo, llegás al otro extremo y la distancia recorrida es exactamente el diámetro.

Algoritmo

1. Elegí un nodo cualquiera (por ejemplo, el 0 o el 1).
2. Hacé un DFS/BFS desde ese nodo y encontrá el vértice más lejano u .
3. Hacé otro DFS/BFS desde u y encontrá el vértice más lejano v .
4. La distancia entre u y v es el diámetro.

Complejidad: $\mathcal{O}(n)$.

Código en C++

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

vector<vector<int>> g;
int mxDist, farthest;

void dfs(int v, int p, int d){
    if(d > mxDist){
        mxDist = d;
        farthest = v;
    }

    for(int u : g[v]){
        if(u != p)
            dfs(u, v, d + 1);
    }
}
```

```

int main(){
    int n;
    cin >> n;

    g.assign(n, {});

    for(int i = 0; i < n - 1; i++){
        int u, v;
        cin >> u >> v;
        --u; --v;

        g[u].push_back(v);
        g[v].push_back(u);
    }

    mxDist = -1;
    dfs(0, -1, 0);
    int extremo1 = farthest;

    mxDist = -1;
    dfs(extremo1, -1, 0);
    int extremo2 = farthest;

    cout << "Diametro = " << mxDist << "\n";
    cout << "Extremos: " << extremo1 + 1 << " " << extremo2 + 1 << "\n";
}

```

Listing 6.28: Obtener diámetro de un árbol

Recuperar el camino del diámetro

Si además quieres recuperar el camino del diámetro, guardá el padre en el segundo DFS.

```

vector<int> parent;

void dfs(int v, int p, int d){
    parent[v] = p;

    if(d > mxDist){
        mxDist = d;
        farthest = v;
    }

    for(int u : g[v]){
        if(u != p)
            dfs(u, v, d + 1);
    }
}

```

Listing 6.29: DFS guardando padres

Después del segundo DFS:

```
vector<int> path;
for(int x = extremo2; x != -1; x = parent[x])
    path.push_back(x);

reverse(path.begin(), path.end());
```

Listing 6.30: Reconstruir camino del diámetro

Ahora `path` contiene todos los nodos del diámetro en orden.

Variante con pesos

Si las aristas tienen pesos positivos, el algoritmo es exactamente el mismo, pero en lugar de sumar 1 en el DFS hacés:

```
dfs(u, v, dist + peso);
```

Listing 6.31: DFS con pesos

La complejidad sigue siendo $\mathcal{O}(n)$ porque el grafo es un árbol.

6.15. Orden Topológico

Se te proporciona un grafo dirigido con n vértices y m aristas. Tienes que encontrar un orden de los vértices, de modo que cada arista vaya desde el vértice con un índice menor hasta un vértice con un índice mayor.

En otras palabras, se trata de encontrar una permutación de los vértices (orden topológico) que corresponda al orden definido por todas las aristas del grafo. Un orden topológico puede no existir en absoluto. Solo existe si el grafo dirigido no contiene ciclos (DAG).

Ejemplo de uso

Un problema común en el que se produce la ordenación topológica es el siguiente. Hay n variables con valores desconocidos. Para algunas variables, sabemos que una de ellas es menor que la otra. Debes comprobar si estas restricciones son contradictorias y, si no lo son, mostrar las variables en orden ascendente (si hay varias respuestas posibles, mostrar cualquiera de ellas). Es fácil notar que este es exactamente el problema de encontrar el orden topológico de un grafo con n vértices.

¿Cómo funciona con DFS?

Supongamos que el grafo es acíclico. ¿Qué hace la búsqueda en profundidad (DFS)?

Al comenzar desde algún vértice v , DFS intenta recorrer a lo largo de todas las aristas que salen de v . Se detiene en los bordes cuyos extremos ya han sido visitados previamente, y recorre el resto de los bordes y continúa recursivamente en sus extremos.

Por lo tanto, en el momento en que la llamada a la función `dfs(v)` ha terminado, todos los vértices que son alcanzables desde v han sido visitados por la búsqueda, ya sea directamente (a través de una arista) o indirectamente.

Añadamos el vértice v a una lista cuando terminemos $\text{dfs}(v)$. Como todos los vértices alcanzables ya han sido visitados, ya estarán en la lista cuando agreguemos v . Hagamos esto para cada vértice del grafo, con una o varias búsquedas en profundidad. Para cada arista dirigida $v \rightarrow u$ en el gráfico, u aparecerá antes en esta lista que v , porque u es accesible desde v .

Entonces, si simplemente etiquetamos los vértices en esta lista con $n-1, n-2, \dots, 1, 0$, hemos encontrado un orden topológico del grafo. En otras palabras, la lista representa el orden topológico inverso, por lo que basta con invertir el arreglo al final.

Código en C++

Aquí se presenta una implementación que asume que el grafo es acíclico, es decir, que existe el orden topológico deseado:

```
// n = numero de vertices
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> ans;

void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) {
        if (!visited[u]) {
            dfs(u);
        }
    }
    ans.push_back(v);
}

void topological_sort() {
    visited.assign(n, false);
    ans.clear();
    for (int i = 0; i < n; ++i) {
        if (!visited[i]) {
            dfs(i);
        }
    }
    reverse(ans.begin(), ans.end());
}
```

Listing 6.32: Orden Topológico con DFS

La función principal de la solución es `topological_sort`, que inicializa las variables DFS, lanza DFS y recibe la respuesta en el vector `ans`.

Vale la pena señalar que cuando el grafo **no** es acíclico, el resultado de `topological_sort` aún sería algo significativo en el sentido de que si un vértice u es alcanzable desde el vértice v , pero no al revés, el vértice v siempre aparecerá primero en el array resultante.

6.16. Componentes conexas

Obtener componentes conexas diferentes de un grafo

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> adj;
vector<bool> vis;

void dfs(int u, vector<int>& componente) {
    vis[u] = true;
    componente.push_back(u);
    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs(v, componente);
        }
    }
}

int main() {
    int n, m;
    cin >> n >> m;

    adj.resize(n);
    vis.assign(n, false);

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        // si los nodos están numerados de 0 a n-1
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<vector<int>> componentes;
    for (int i = 0; i < n; i++) {
        if (!vis[i]) {
            vector<int> componente;
            dfs(i, componente);
            componentes.push_back(componente);
        }
    }

    cout << "Cantidad de componentes: " << componentes.size() << "\n\n";
    for (size_t i = 0; i < componentes.size(); i++) {
        cout << "Componente " << i + 1 << ": ";
        for (int nodo : componentes[i]) {
            cout << nodo << " ";
        }
        cout << "\n";
    }
}
```

```
}
}
```

Listing 6.33: Código para obtener componentes conexas

Ejemplo

Entrada:

```
7 4
0 1
1 2
3 4
5 6
```

Salida posible:

```
Cantidad de componentes: 3
Componente 1: 0 1 2
Componente 2: 3 4
Componente 3: 5 6
```

Maapeo rápido de nodo a componente

Se puede mantener un vector `vector<int>comp(n, -1)`; y asignar dentro del DFS:

```
comp[u] = id_componente;
```

Listing 6.34: Asignación de componente

Esto permite conocer en $O(1)$ a qué componente pertenece cada vértice, útil en muchos problemas de programación competitiva.

6.17. Matching (Algoritmo Húngaro)

¿Para qué sirve el algoritmo Húngaro?

El algoritmo húngaro resuelve el *problema de asignación*: dadas n tareas y n trabajadores (o n filas y n columnas de una matriz de costos), se desea asignar cada tarea a un trabajador de forma que se minimice la suma total de los costos. En términos de grafos, es encontrar un *matching perfecto* de peso mínimo en un grafo bipartito completo.

Este problema aparece frecuentemente en programación competitiva, por ejemplo para:

- Asignar trabajos a máquinas minimizando el tiempo total.
- Emparejar puntos de dos imágenes minimizando la suma de distancias.
- Resolver sub-problemas de coloreado de árboles donde los colores de los hijos se asignan mediante un coste.

Cómo funciona el algoritmo

El algoritmo mantiene dos potenciales $u[i]$ (filas) y $v[j]$ (columnas) con la condición $u[i] + v[j] \leq A[i][j]$. Las aristas donde la igualdad se cumple son rígidas y forman un grafo H . Se busca incrementar el tamaño del matching máximo de H usando caminos aumentantes (Kuhn). Si no hay camino aumentante, se reajusta el potencial aumentando u y disminuyendo v en la cantidad mínima Δ que permite crear nuevas aristas rígidas. Repetimos hasta obtener un matching perfecto. La versión optimizada tiene complejidad $O(n^3)$.

Implementación en C++ (versión compacta)

```
#include <bits/stdc++.h>
using namespace std;

#define fore(i,a,b) for(ll i=(a);i<(b);i++)
#define all(x) (x).begin(), (x).end()
#define pb push_back
#define fst first
#define snd second
#define sz(x) ((int)x.size())
#define FIN ios::sync_with_stdio(0);cin.tie(0);cout.tie(0);

using ll = long long;
typedef vector<double> vi;

int n, k;
vector<vector<double>> g;
vector<double> mt;
vector<bool> used;

const double INF = 1e9;

pair<double, vector<int>> hungarian(const vector<vector<double>>& cost) {
    int n = cost.size();
    int m = cost[0].size();
    vector<double> u(n + 1, 0), v(m + 1, 0);
    vector<int> p(m + 1, 0), ans(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        int j0 = 0;
        vector<double> min_v(m + 1, INF);
        vector<int> used(m + 1, 0);
        vector<int> way(m + 1, 0);
        do {
            used[j0] = 1;
            int i0 = p[j0];
            int j1 = 0;
            double delta = INF;
            for (int j = 1; j <= m; ++j) if (!used[j]) {
                double cur = cost[i0-1][j-1] - u[i0] - v[j];
```

```

        if (cur < min_v[j]) { min_v[j] = cur; way[j] = j0; }
        if (min_v[j] < delta) { delta = min_v[j]; j1 = j; }
    }
    for (int j = 0; j <= m; ++j) {
        if (used[j]) { u[p[j]] += delta; v[j] -= delta; }
        else min_v[j] -= delta;
    }
    j0 = j1;
} while (p[j0] != 0);
do {
    int j1 = way[j0];
    p[j0] = p[j1];
    j0 = j1;
} while (j0);
}
vector<int> matching(n);
for (int j = 1; j <= m; ++j) if (p[j]) matching[p[j]-1] = j-1;
double total_cost = -v[0];
return {total_cost, matching};
}

int main(){
    FIN;
    ll n; cin>>n;
    g.resize(n, vector<double>(n));
    fore(i,0,n){
        fore(j,0,n){
            ll num; cin>>num;
            double num2 = log(num);
            num2 *= -1;
            g[i][j] = num2;
        }
    }
    auto [min_cost, assignment] = hungarian(g);
    vector<pair<int,int>> res;
    for (int i = 0; i < (int)assignment.size(); ++i) {
        res.push_back({assignment[i] + 1, i + 1}); // (columna, fila)
    }
    sort(all(res));
    for (auto [col, fila] : res) {
        cout << fila << " ";
    }
    cout << "\n";
    return 0;
}

```

Listing 6.35: Implementación del algoritmo Húngaro

Explicación paso a paso

1. **Lectura de la matriz de costos:** en el programa se lee una matriz $n \times n$ y se transforma cada valor c en $-\log(c)$, para convertir un problema de **maximizar producto** en uno de **minimizar suma**.
2. **Llamada a hungarian:** la función devuelve el coste mínimo y el vector `assignment` donde `assignment[i]` indica la columna asignada a la fila i .
3. **Construcción de la salida:** se crea un vector de pares (`columna`, `fila`), se ordena por columna y se imprimen las filas en orden, tal como se suele requerir en concursos.

Cuándo usarlo

- Cuando el número de filas y columnas es pequeño a mediano ($n \leq 500$ suele ser cómodo).
- Cuando se necesita la solución exacta del problema de asignación, no una heurística.
- En problemas donde el coste se define por distancias euclidianas, log-probabilidades, pesos de aristas, etc.

El algoritmo húngaro es una herramienta esencial en la caja de trucos de cualquier competidor de programación competitiva.

Capítulo 7

Strings

7.1. Permutaciones de string

```
int main() {
    string s = "ABA";

    IMPORTANTE: Ordenar la cadena primero
    sort(s.begin(), s.end());

    El bucle do-while asegura que se procese la 1 permutacion
    do {
        cout << s << endl;
    } while (next_permutation(s.begin(), s.end()));

    return 0;
}
```

Listing 7.1: Next_permutation

Complejidad: Para una cadena de longitud n es $n!$.

7.2. Palindromic Tree (Eertree)

7.2.1. ¿Qué es?

El **Palindromic Tree** (también llamado *Eertree*) es una estructura de datos que representa **todos los substrings palíndromos distintos** de un string s de longitud n . Tiene a lo sumo $n+2$ nodos: uno por cada palíndromo distinto, más dos raíces imaginarias.

¿Cuándo usarlo?

- Contar la cantidad de substrings palíndromos distintos.
- Contar cuántas veces aparece cada palíndromo como substring.
- Problemas que requieren explotar la estructura de palíndromos (DP sobre palíndromos, etc.).

7.2.2. Estructura interna

Cada nodo del árbol representa un palíndromo distinto que aparece en s .

- **len**: longitud del palíndromo que representa este nodo.
- **link**: *suffix link* — apunta al nodo del palíndromo sufijo propio más largo de este nodo.
- **to[c]**: transición: si al nodo v le agregamos el carácter c a ambos lados, llegamos al nodo **to[c]**.
- **cnt**: cantidad de veces que este palíndromo aparece como sufijo al procesar el string (se propaga al final para obtener ocurrencias totales).

Raíces especiales:

- **Nodo 0**: raíz imaginaria con **len** = -1 (palíndromo de longitud -1, sirve para el caso base).
- **Nodo 1**: raíz imaginaria con **len** = 0 (palíndromo vacío).

7.2.3. Implementación

```
struct palindromic_tree {
    static const int SIGMA = 26;
    struct Node {
        int len, link, to[SIGMA];
        ll cnt;
        Node(int len, int link = 0, ll cnt = 1)
            : len(len), link(link), cnt(cnt) {
            memset(to, 0, sizeof(to));
        }
    };
    vector<Node> ns;
    int last;

    palindromic_tree() : last(0) {
        ns.pb(Node(-1)); // nodo 0: raíz imaginaria len=-1
        ns.pb(Node(0));  // nodo 1: raíz vacía    len=0
    }

    void add(int i, string& s) {
        int p = last, c = s[i] - 'a';
        // Buscar el sufijo palindromo mas largo al que se puede
        // agregar s[i] a la izquierda
        while (s[i - ns[p].len - 1] != s[i]) p = ns[p].link;
        if (ns[p].to[c]) {
            // El palindromo ya existe: solo actualizamos last
            last = ns[p].to[c];
            ns[last].cnt++;
        } else {
            // Crear un nuevo nodo para el nuevo palindromo
            int q = ns[p].link;
            while (s[i - ns[q].len - 1] != s[i]) q = ns[q].link;
            q = max(1, ns[q].to[c]); // suffix link del nuevo nodo
        }
    }
};
```

```

        last = ns[p].to[c] = (int)ns.size();
        ns.pb(Node(ns[p].len + 2, q, 1));
    }
}
};

```

Listing 7.2: Palindromic Tree (Eertree)

7.2.4. ¿Qué tengo que pasarle?

Se llama `add(i, s)` para cada posición i del string s , de izquierda a derecha.

Importante: el string s debe tener un centinela al inicio (posición 0) que no aparezca en el alfabeto, por ejemplo '\$'. Esto evita que el `while` salga de rango.

```

string t; cin >> t;
string s = "$" + t; // centinela en posicion 0

palindromic_tree pt;
for (int i = 1; i < (int)s.size(); i++)
    pt.add(i, s);

```

Listing 7.3: Uso basico del Palindromic Tree

7.2.5. ¿Qué puedo consultar?

Cantidad de palíndromos distintos

La cantidad de substrings palíndromos distintos es la cantidad de nodos del árbol menos 2 (las dos raíces imaginarias):

```

int distintos = (int)pt.ns.size() - 2;

```

Listing 7.4: Contar palindromos distintos

Ocurrencias de cada palíndromo

Después de procesar todo el string, `ns[v].cnt` solo cuenta las veces que el palíndromo aparece como *sufijo* al momento de ser agregado. Para obtener las ocurrencias totales como substring hay que propagar los `cnt` por los suffix links, de mayor a menor longitud (los nodos están en orden topológico inverso):

```

// Propagar de atras para adelante (orden topologico)
for (int i = (int)pt.ns.size() - 1; i >= 2; i--)
    pt.ns[pt.ns[i].link].cnt += pt.ns[i].cnt;

// Ahora pt.ns[v].cnt = cantidad total de apariciones del palindromo v

```

Listing 7.5: Propagar cnt para obtener ocurrencias totales

7.2.6. Ejemplo de aplicación

Problema: dado un string, imprimir la longitud y cantidad de ocurrencias de cada substring palíndromo distinto.

```

string t; cin >> t;
string s = "$" + t;

palindromic_tree pt;
for (int i = 1; i < (int)s.size(); i++)
    pt.add(i, s);

// Propagar ocurrencias
for (int i = (int)pt.ns.size() - 1; i >= 2; i--)
    pt.ns[pt.ns[i].link].cnt += pt.ns[i].cnt;

// Imprimir cada palindromo (nodos 2 en adelante)
for (int i = 2; i < (int)pt.ns.size(); i++)
    cout << "len=" << pt.ns[i].len
         << " ocurrencias=" << pt.ns[i].cnt << "\n";

```

Listing 7.6: Imprimir todos los palindromos con su frecuencia

Ejemplo: para `s = .abaab`:

- `.a` (len=1): 3 ocurrencias
- `"b"` (len=1): 2 ocurrencias
- `.aba` (len=3): 1 ocurrencia
- `.aa` (len=2): 1 ocurrencia
- `"baab"` (len=4): 1 ocurrencia

7.2.7. Complejidad

- **Tiempo:** $O(n \cdot \Sigma)$ para construcción, donde Σ es el tamaño del alfabeto.
- **Espacio:** $O(n \cdot \Sigma)$ para el árbol.
- **Nodos:** a lo sumo $n + 2$ nodos (uno por cada palíndromo distinto).

Capítulo 8

Estructuras de Datos

8.1. Union Find / Disjoint Set Union (DSU)

El **Union-Find** es una estructura de datos que permite mantener un conjunto de elementos particionados en subconjuntos disjuntos. Es muy eficiente para operaciones de conectividad, ya que nos dice si dos elementos pertenecen al mismo grupo y nos permite unirlos. Se basa en dos operaciones principales:

- **find(x)**: Sirve para saber a qué conjunto pertenece el elemento x . Devuelve el representante" (o raíz) de ese conjunto.
- **unite(a, b)** o **union(a, b)**: Une los conjuntos a los que pertenecen a y b .

Uso en grillas 2D

El Union-Find está diseñado para trabajar con estructuras en una sola dimensión (1D), es decir, maneja elementos como si fueran posiciones en un arreglo lineal $(0, 1, 2, \dots, N - 1)$. Por eso, cuando el problema es en 2D, como una grilla de filas y columnas (r, c) , primero hay que convertir cada celda en un único número para poder usar DSU correctamente.

Esto se hace con la siguiente fórmula:

$$id = r \times m + c$$

(Donde m es la cantidad de columnas de la grilla).

```
struct DSU {
    vector<int> parent;
    vector<int> sz; // Tamano del componente

    DSU(int n) {
        parent.resize(n + 1);
        sz.assign(n + 1, 1);
        for (int i = 0; i <= n; i++) parent[i] = i;
    }

    // Encuentra el representante con Compresion de Caminos
    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]);
    }
}
```

```

// Une dos conjuntos por Tamano
void unite(int i, int j) {
    int root_i = find(i);
    int root_j = find(j);
    if (root_i != root_j) {
        if (sz[root_i] < sz[root_j]) swap(root_i, root_j);
        parent[root_j] = root_i;
        sz[root_i] += sz[root_j];
    }
}

bool same(int i, int j) {
    return find(i) == find(j);
}
};

```

8.2. Fenwick Tree (BIT)

El **Fenwick Tree** o **Binary Indexed Tree (BIT)** es una estructura de datos sobre un arreglo de números que permite mantener información acumulada y responder consultas de forma eficiente.

Se usa principalmente para dos operaciones:

- **Actualización de punto:** sumar un valor a una posición específica.
- **Consulta de prefijo:** obtener la suma de los elementos desde la posición 1 hasta i .

Complejidad temporal

- update: $O(\log n)$
- query: $O(\log n)$

Complejidad espacial: $O(n)$.

Idea principal

Cada posición del BIT guarda la suma de un bloque de elementos del arreglo original. Gracias a esto, para actualizar o consultar no es necesario recorrer todo el arreglo: solo se visitan algunos nodos relacionados con el bit menos significativo de cada índice.

Función `update(idx, val)`

Sirve para sumar `val` en la posición `idx`.

En la práctica, el arreglo del BIT y el tamaño `n` suelen declararse como variables globales o como atributos de una estructura/clase, porque las funciones `update` y `query` necesitan acceder a ellos. No es obligatorio que sean globales: también se pueden pasar como parámetros o encapsular dentro de un `struct`, pero en programación competitiva se usa mucho la versión global por simplicidad.

- **Cómo funciona:** empieza en `idx` y va subiendo hacia sus "padres" en la estructura, agregando el valor en cada bloque afectado.
- **Importante:** este BIT normalmente trabaja con índices desde 1 hasta n .

```
void update(int idx, int val) {
    for (; idx <= n; idx += idx & -idx)
        bit[idx] += val;
}
```

Función query(idx)

Calcula la suma de los elementos desde la posición 1 hasta `idx`.

- **Cómo funciona:** suma el valor en `idx` y luego retrocede eliminando el bit menos significativo hasta llegar a cero.

```
int query(int pos) {
    int res = 0;
    for (; pos > 0; pos -= pos & -pos)
        res += bit[pos];
    return res;
}
```

Para obtener la suma de un rango $[L, R]$, se usa:

$$query(R) - query(L - 1)$$

¿En qué tipos de problemas sirve?

El BIT es útil cuando necesitas una mezcla de consultas y modificaciones en tiempo real, sobre todo si ambas afectan un solo punto del arreglo.

- **Cálculo de inversiones:** contar pares (i, j) tales que $i < j$ y $A[i] > A[j]$.
- **Actualizaciones de punto y consultas de rango:** cuando el arreglo cambia constantemente y necesitas responder sumas rápidas.
- **Conteo de frecuencias:** mantener cuántas veces aparece cada valor para responder consultas sobre prefijos o rangos de frecuencias.

Observación

Si el problema requiere actualizaciones en un rango completo o consultas más complejas, a veces conviene usar un segment tree; si solo necesitas sumas de prefijos con cambios puntuales, el Fenwick Tree suele ser más simple y rápido de implementar.

Capítulo 9

Arrays / Subarrays

9.1. Kadane

Kadane es un algoritmo para encontrar el subarreglo contiguo con mejor suma en tiempo lineal.

¿Qué hace? Recorre el arreglo una sola vez y mantiene:

- **cur**: la mejor suma que termina en la posición actual
- **best**: la mejor suma global encontrada hasta el momento

Para cada elemento **x**, decide entre:

- empezar un subarreglo nuevo en **x**
- extender el subarreglo anterior (**cur + x**)

```
long long kadane_max(const vector<long long>& arr) {
    long long cur = arr[0], best = arr[0];
    for (int i = 1; i < (int)arr.size(); ++i) {
        cur = max(arr[i], cur + arr[i]);
        best = max(best, cur);
    }
    return best;
}
```

Listing 9.1: Kadane para subarreglo de suma maxima

Cómo usarlo para buscar el mínimo La idea es la misma, pero reemplazando max por min.

Complejidad Tanto **kadane_max** como **kadane_min** tienen:

- **Tiempo**: $O(n)$
- **Memoria extra**: $O(1)$

9.1.1. Kadane con restricción (el subarreglo debe incluir k)

Si queremos la suma máxima de un subarreglo contiguo que **obligatoriamente** contenga la posición **k**, podemos precomputar:

- **left[i]**: mejor suma de un subarreglo que termina en **i**

- `right[i]`: mejor suma de un subarreglo que empieza en `i`

Entonces, la mejor respuesta que incluye `k` es:

$$\text{left}[k] + \text{right}[k] - a[k]$$

```
long long kadane_con_k_max(const vector<long long>& a, int k) {
    int n = (int)a.size();
    vector<long long> left(n), right(n);

    left[0] = a[0];
    for (int i = 1; i < n; i++) {
        left[i] = max(a[i], left[i - 1] + a[i]);
    }

    right[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        right[i] = max(a[i], right[i + 1] + a[i]);
    }

    return left[k] + right[k] - a[k];
}
```

Listing 9.2: Kadane maximo forzando incluir la posicion `k` (0-based)

Complejidad de esta variante

- **Tiempo:** $O(n)$
- **Memoria extra:** $O(n)$ por los arreglos `left` y `right`